Datové struktury

Lineární struktury a stromy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Co jsou datové struktury?

Datová struktura je

Co jsou datové struktury?

Datová struktura je způsob organizace a ukládání dat v počítači

Co jsou datové struktury?

Datová struktura je způsob organizace a ukládání dat v počítači

- Definuje vztahy mezi daty

Co jsou datové struktury?

Datová struktura je způsob organizace a ukládání dat v počítači

- Definuje vztahy mezi daty
- Určuje operace, které lze s daty provádět

Co jsou datové struktury?

Datová struktura je způsob organizace a ukládání dat v počítači

- Definuje vztahy mezi daty
- Určuje operace, které lze s daty provádět
- Ovlivňuje **efektivitu** algoritmů

Co jsou datové struktury?

Datová struktura je způsob organizace a ukládání dat v počítači

- Definuje vztahy mezi daty
- Určuje operace, které lze s daty provádět
- Ovlivňuje **efektivitu** algoritmů
- Volba správné datové struktury = rychlejší a úspornější program

Co jsou datové struktury?

Představte si knihovnu

Co jsou datové struktury?

Představte si knihovnu

Bez organizace:

- Knihy v náhodném pořadí
- Hledání knihy trvá hodiny

Co jsou datové struktury?

Představte si knihovnu

Bez organizace:

- Knihy v náhodném pořadí
- Hledání knihy trvá hodiny

S organizací:

- Knihy seřazené podle autora/žánru
- Rychlé vyhledání pomocí katalogu

Proč jsou datové struktury důležité?

Jaké výhody podle vás přináší správná volba datové struktury?

Proč jsou datové struktury důležité?

Jaké výhody podle vás přináší správná volba datové struktury?

- **Optimalizace času** - rychlejší operace

Proč jsou datové struktury důležité?

Jaké výhody podle vás přináší správná volba datové struktury?

- **Optimalizace času** - rychlejší operace
- **Optimalizace paměti** - úsporné ukládání dat

Proč jsou datové struktury důležité?

Jaké výhody podle vás přináší správná volba datové struktury?

- **Optimalizace času** - rychlejší operace
- **Optimalizace paměti** - úsporné ukládání dat
- **Řešení specifických problémů** - každá struktura pro jiný účel

Proč jsou datové struktury důležité?

Jaké výhody podle vás přináší správná volba datové struktury?

- **Optimalizace času** - rychlejší operace
- **Optimalizace paměti** - úsporné ukládání dat
- **Řešení specifických problémů** - každá struktura pro jiný účel
- **Základ pro algoritmy** - mnoho algoritmů závisí na datových strukturách

Proč jsou datové struktury důležité?

Příklad - vyhledávání v datech:

```
// Array with 1,000,000 elements
const array: number[] = [1, 2, 3, ..., 1000000];

// Linear search - average 500,000 steps
array.find((x: number) => x === 999999);
```

Proč jsou datové struktury důležité?

Příklad - vyhledávání v datech:

```
// Array with 1,000,000 elements
const array: number[] = [1, 2, 3, ..., 1000000];

// Linear search - average 500,000 steps
array.find((x: number) => x === 999999);

// Binary tree - only ~20 steps!
binarySearchTree.find(999999);
```

Zásobník (Stack)

Zásobník

Zásobník je

Zásobník

Zásobník je datová struktura založená na principu

Zásobník

Zásobník je datová struktura založená na principu **LIFO**

Zásobník

Zásobník je datová struktura založená na principu **LIFO**

LIFO =

Zásobník

Zásobník je datová struktura založená na principu **LIFO**

LIFO = **Last In, First Out**

Zásobník

Zásobník je datová struktura založená na principu **LIFO**

LIFO = **Last In, First Out**

Poslední přidaný prvek je první odebraný

Zásobník

Představte si naskládané talíře

Zásobník

Představte si naskládané talíře

- Nové talíře se přidávají na

Zásobník

Představte si naskládané talíře

- Nové talíře se přidávají na vrchol

Zásobník

Představte si naskládané talíře

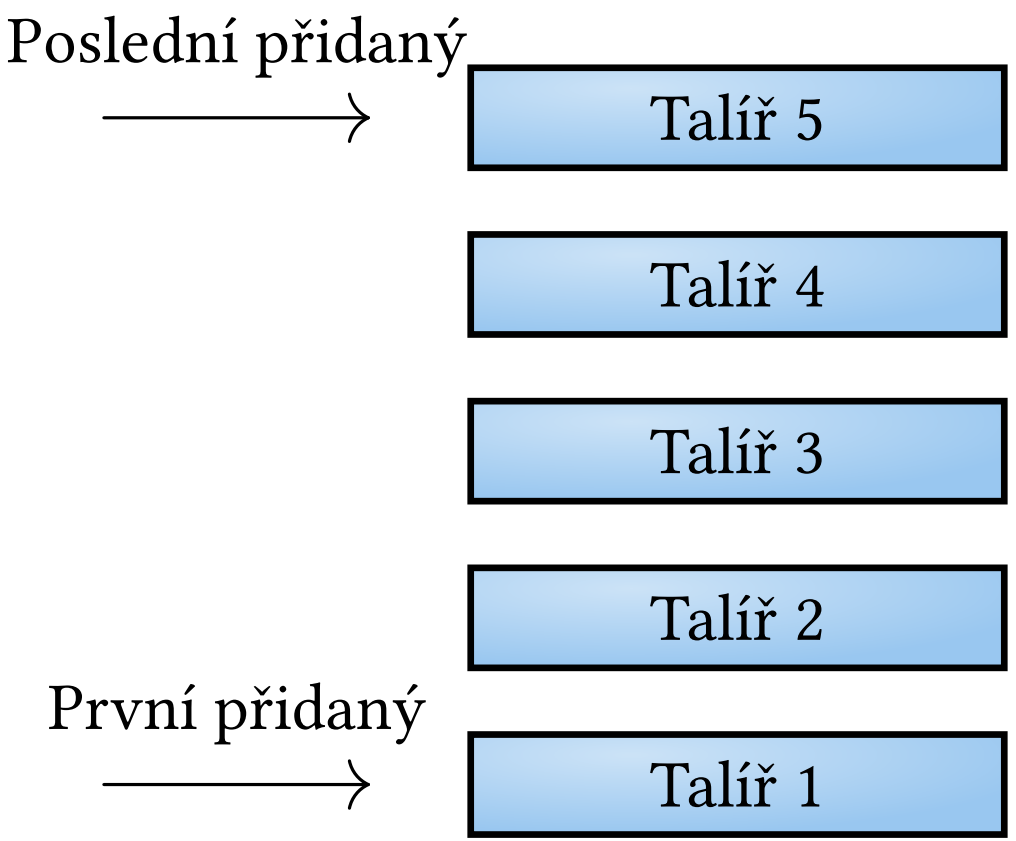
- Nové talíře se přidávají na vrchol
- Talíře se odebírají z

Zásobník

Představte si naskládané talíře

- Nové talíře se přidávají na vrchol
- Talíře se odebírají z vrcholu

Zásobník


Poslední přidaný
Talíř 5
Talíř 4
Talíř 3
Talíř 2
První přidaný
Talíř 1

Zásobník

Základní operace:

Zásobník

Základní operace:

- `push(item)` - přidá prvek na vrchol

Zásobník

Základní operace:

- `push(item)` - přidá prvek na vrchol
- `pop()` - odebere a vrátí prvek z vrcholu

Zásobník

Základní operace:

- `push(item)` - přidá prvek na vrchol
- `pop()` - odebere a vrátí prvek z vrcholu
- `peek()` - zobrazí prvek na vrcholu (bez odebrání)

Zásobník

Základní operace:

- `push(item)` - přidá prvek na vrchol
- `pop()` - odebere a vrátí prvek z vrcholu
- `peek()` - zobrazí prvek na vrcholu (bez odebrání)
- `isEmpty()` - kontrola, zda je zásobník prázdný

Implementace v TypeScriptu

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void { this.items.push(item); }

  pop(): T | undefined { return this.items.pop(); }

  peek(): T | undefined { return this.items.at(-1); }

  isEmpty(): boolean { return this.items.length === 0; }
}
```

Call stack - Implicitní zásobník

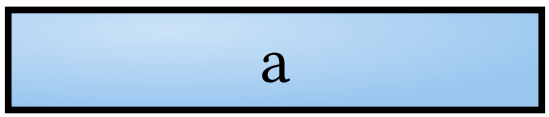
```
function a(): void { b(); }

function b(): void { c(); }

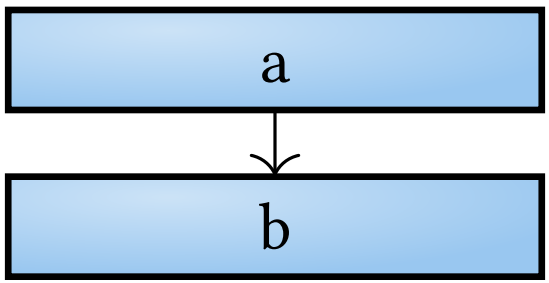
function c(): void { console.log("Done"); }

a();
```

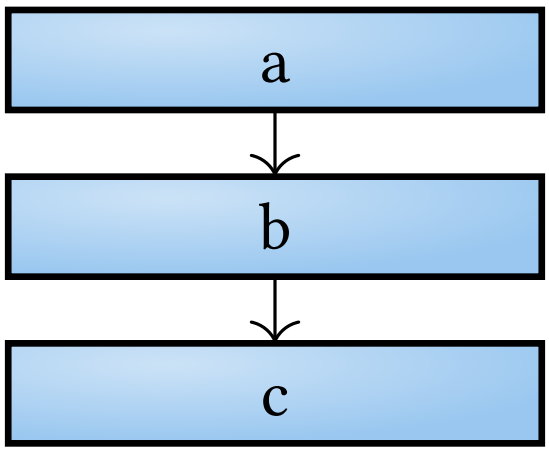
Call stack - Implicitní zásobník



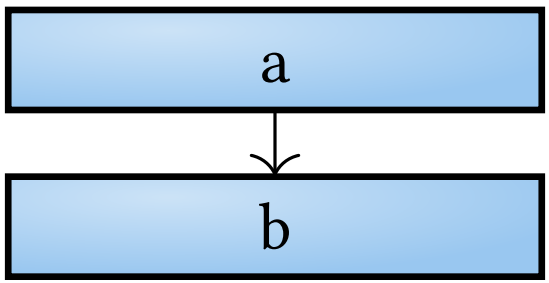
Call stack - Implicitní zásobník



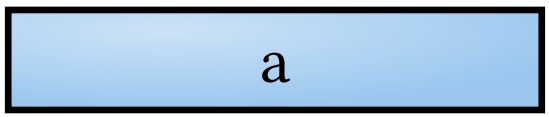
Call stack - Implicitní zásobník



Call stack - Implicitní zásobník



Call stack - Implicitní zásobník



Příklad - Validace závorek

```
checkBrackets("({[]})");  // true
checkBrackets("({[}])");  // false

function checkBrackets(str: string): boolean {
  const stack = new Stack<string>();
  const pairs: Record<string, string> = {
    '(': ')', '[': ']', '{': '}'
  };

  // TODO: Live implementation now
```

Příklad - Validace závorek

```
  for (const char of str) {
    if (char in pairs) {
      stack.push(char);  // Opening bracket
    } else if (Object.values(pairs).includes(char)) {
      const last = stack.pop();
      if (!last || pairs[last] !== char) return false;
    }
  }
  return stack.isEmpty();
}
```

Další příklady zásobníku

- **Undo/Redo** operace

Další příklady zásobníku

- **Undo/Redo** operace
- **Procházení do hloubky** (DFS) v grafech

Další příklady zásobníku

- **Undo/Redo** operace
- **Procházení do hloubky** (DFS) v grafech
- **Parsování** výrazů a vyhodnocování postfixové notace

Další příklady zásobníku

- **Undo/Redo** operace
- **Procházení do hloubky** (DFS) v grafech
- **Parsování** výrazů a vyhodnocování postfixové notace
- **Překladače** a **interpreti** programovacích jazyků

Další příklady zásobníku

- **Undo/Redo** operace
- **Procházení do hloubky** (DFS) v grafech
- **Parsování** výrazů a vyhodnocování postfixové notace
- **Překladače** a **interpreti** programovacích jazyků
- **Historie** prohlížeče (tlačítko Zpět)

Fronta (Queue)

Fronta

Fronta je

Fronta

Fronta je datová struktura založená na principu

Fronta

Fronta je datová struktura založená na principu **FIFO**

Fronta

Fronta je datová struktura založená na principu **FIFO**

FIFO = First In, First Out

Fronta

Fronta je datová struktura založená na principu **FIFO**

FIFO = First In, First Out

První přidaný prvek je první odebraný

Fronta

Představte si frontu v obchodě

Fronta

Představte si frontu v obchodě

- Noví zákazníci se přidávají na konec fronty

Fronta

Představte si frontu v obchodě

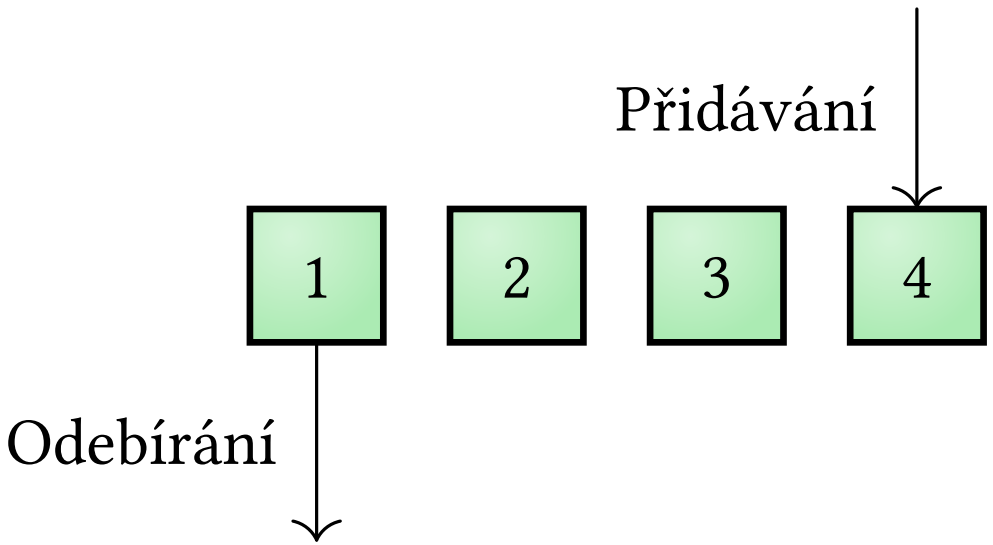
- Noví zákazníci se přidávají na konec fronty
- Zákazníci jsou obslouženi v pořadí příchodu

Fronta

Představte si frontu v obchodě

- Noví zákazníci se přidávají na konec fronty
- Zákazníci jsou obslouženi v pořadí příchodu

Fronta



Fronta

Základní operace:

Fronta

Základní operace:

- `enqueue(item)` - přidá prvek na konec fronty

Fronta

Základní operace:

- `enqueue(item)` - přidá prvek na konec fronty
- `dequeue()` - odebere a vrátí prvek ze začátku fronty

Fronta

Základní operace:

- `enqueue(item)` - přidá prvek na konec fronty
- `dequeue()` - odebere a vrátí prvek ze začátku fronty
- `peek()` - zobrazí první prvek (bez odebrání)

Fronta

Základní operace:

- `enqueue(item)` - přidá prvek na konec fronty
- `dequeue()` - odebere a vrátí prvek ze začátku fronty
- `peek()` - zobrazí první prvek (bez odebrání)
- `isEmpty()` - kontrola, zda je fronta prázdná

Implementace v TypeScriptu

```
class Queue<T> {
  private items: T[] = [];

  enqueue(item: T): void { this.items.push(item); }

  dequeue(): T | undefined { return this.items.shift(); }

  peek(): T | undefined { return this.items[0]; }

  isEmpty(): boolean { return this.items.length === 0; }
}
```

Typy front

Jednoduchá fronta - základní FIFO

Typy front

Jednoduchá fronta - základní FIFO

Kruhová fronta (Circular Queue) - konec se napojuje na začátek

Typy front

Jednoduchá fronta - základní FIFO

Kruhová fronta (Circular Queue) - konec se napojuje na začátek

Prioritní fronta (Priority Queue) - prvky mají priority

Typy front

Jednoduchá fronta - základní FIFO

Kruhová fronta (Circular Queue) - konec se napojuje na začátek

Prioritní fronta (Priority Queue) - prvky mají priority

Fronta s dvěma konci (Deque) - přidávání a odebírání z obou stran

Příklad - Správa úloh

```
// Task queue for async processing
type Task = () => Promise<void>;

class TaskQueue {
  private queue = new Queue<Task>();
  private processing = false;
```

Příklad - Správa úloh

```
class TaskQueue {
  ...
  async addTask(task: Task): Promise<void> {
    this.queue.enqueue(task);
  }
```

Příklad - Správa úloh

```
private async processTasks(): Promise<void> {
  if (this.processing) {
    return;
  }
  this.processing = true;
  while (!this.queue.isEmpty()) {
    const task = this.queue.dequeue();
    if (task) await task();
  }
  this.processing = false;
}
```

Další příklady fronty

- **Správa úloh** v operačních systémech

Další příklady fronty

- **Správa úloh** v operačních systémech
- **Buffery** pro vstup/výstup (I/O)

Další příklady fronty

- **Správa úloh** v operačních systémech
- **Buffery** pro vstup/výstup (I/O)
- **Procházení do šířky** (BFS) v grafech

Další příklady fronty

- **Správa úloh** v operačních systémech
- **Buffery** pro vstup/výstup (I/O)
- **Procházení do šířky** (BFS) v grafech
- **Message queue** v distribuovaných systémech

Další příklady fronty

- **Správa úloh** v operačních systémech
- **Buffery** pro vstup/výstup (I/O)
- **Procházení do šířky** (BFS) v grafech
- **Message queue** v distribuovaných systémech
- **Print spooler** - fronta tiskových úloh

Spojový seznam (Linked List)

Spojový seznam

Spojový seznam je

Spojový seznam

Spojový seznam je kolekce uzlů propojených odkazy

Spojový seznam

Spojový seznam je kolekce uzlů propojených odkazy

Každý uzel obsahuje:

Spojový seznam

Spojový seznam je kolekce uzlů propojených odkazy

Každý uzel obsahuje:

- **Data** (hodnotu)

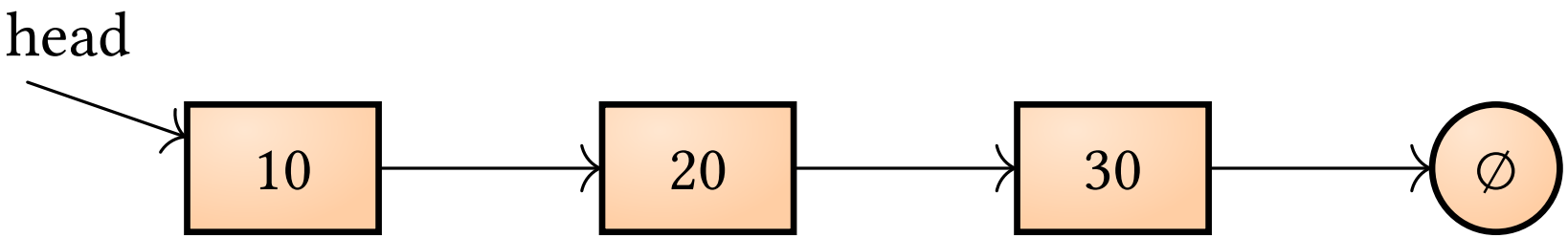
Spojový seznam

Spojový seznam je kolekce uzlů propojených odkazy

Každý uzel obsahuje:

- **Data** (hodnotu)
- **Odkaz** na další uzel (nebo `null`)

Vizualizace



Rozdíly oproti poli

Rozdíly oproti poli

- Uzly **nejsou v paměti vedle sebe**

Rozdíly oproti poli

- Uzly **nejsou v paměti vedle sebe**
  - Časté výpadky cache paměti (cache miss)

Rozdíly oproti poli

- Uzly **nejsou v paměti vedle sebe**
  - Časté výpadky cache paměti (cache miss)
- Přístup pouze **sekvenčně** přes odkazy
  - Nelze přejít na libovolný prvek přímo

Typy spojových seznamů

Jednosměrný (Singly Linked List)

Typy spojových seznamů

Jednosměrný (Singly Linked List)

Každý uzel ukazuje pouze na

Typy spojových seznamů

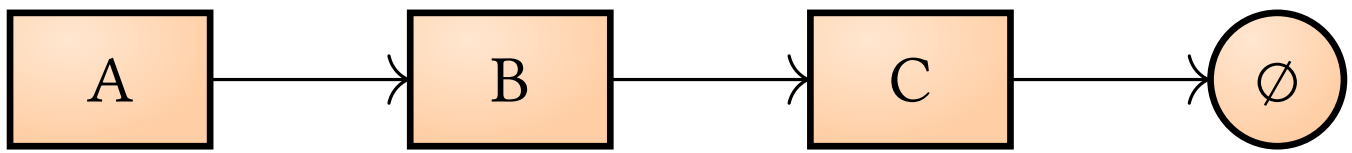
Jednosměrný (Singly Linked List)

Každý uzel ukazuje pouze na **následující** uzel

Typy spojových seznamů

Jednosměrný (Singly Linked List)

Každý uzel ukazuje pouze na **následující** uzel



Typy spojových seznamů

Obousměrný (Doubly Linked List)

Typy spojových seznamů

Obousměrný (Doubly Linked List)

Každý uzel ukazuje na

Typy spojových seznamů

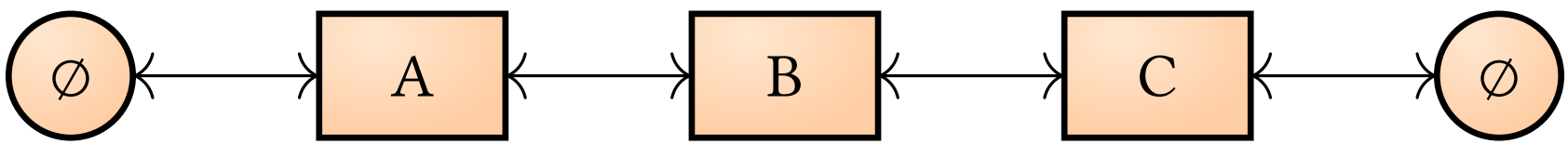
Obousměrný (Doubly Linked List)

Každý uzel ukazuje na **předchozí** i **následující** uzel

Typy spojových seznamů

Obousměrný (Doubly Linked List)

Každý uzel ukazuje na **předchozí** i **následující** uzel



Typy spojových seznamů

Kruhový (Circular Linked List)

Typy spojových seznamů

Kruhový (Circular Linked List)

Poslední uzel ukazuje zpět na

Typy spojových seznamů

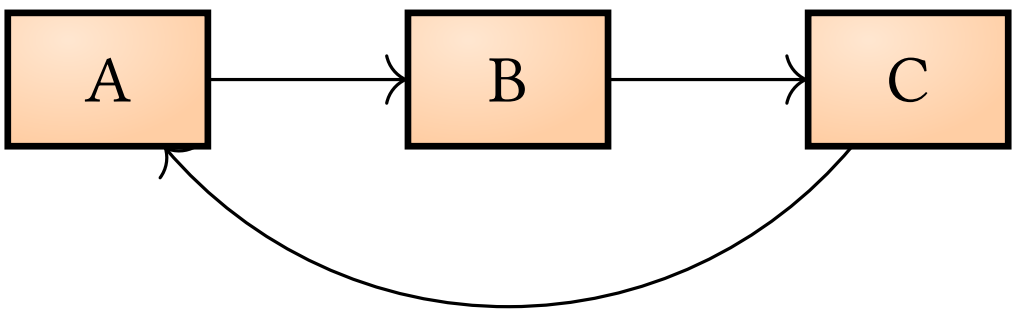
Kruhový (Circular Linked List)

Poslední uzel ukazuje zpět na **první** uzel

Typy spojových seznamů

Kruhový (Circular Linked List)

Poslední uzel ukazuje zpět na **první** uzel



Implementace

```
class Node<T> {
  data: T;
  next: Node<T> | null = null;
  constructor(data: T) {
    this.data = data;
  }
}
```

Implementace

```
class Node<T> {
  data: T;
  next: Node<T> | null = null;
  constructor(data: T) {
    this.data = data;
  }
}

class LinkedList<T> {
  private head: Node<T> | null = null;
  private size = 0;
```

Implementace

Vložení na začátek

Implementace

Vložení na začátek

```
class LinkedList<T> {
  ...
  prepend(data: T): void {
    const newNode = new Node(data);
    newNode.next = this.head;
    this.head = newNode;
    this.size++;
  }
```

Časová složitost:

Implementace

Vložení na začátek

```
class LinkedList<T> {
  ...
  prepend(data: T): void {
    const newNode = new Node(data);
    newNode.next = this.head;
    this.head = newNode;
    this.size++;
  }
```

Časová složitost: $\mathcal{O}(1)$

Implementace

Vložení na konec

Implementace

Vložení na konec

```
class LinkedList<T> {
  ...
  append(data: T): void {
    const newNode = new Node(data);
    if (!this.head) {
      this.head = newNode;
    } else {
```

Implementace

```
      let current = this.head;
      while (current.next) {
        current = current.next;  // Traverse entire list
      }
      current.next = newNode;
    }
    this.size++;
  }
```

Časová složitost:

Implementace

```
      let current = this.head;
      while (current.next) {
        current = current.next;  // Traverse entire list
      }
      current.next = newNode;
    }
    this.size++;
  }
```

Časová složitost: $\mathcal{O}(n)$

Implementace

Odstranění prvku

Implementace

Odstranění prvku

```
class LinkedList<T> {
  ...
  remove(data: T): void {
    if (!this.head) return;

    if (this.head.data === data) { // Removing head
      this.head = this.head.next;
      this.size--;
      return;
    }
```

Implementace

```
    let current = this.head;
    while (current.next) { // Traverse entire list
      if (current.next.data === data) {
        current.next = current.next.next; this.size--;
        return;
      }
      current = current.next;
    }
  }
}
```

Časová složitost:

Implementace

```
    let current = this.head;
    while (current.next) { // Traverse entire list
      if (current.next.data === data) {
        current.next = current.next.next; this.size--;
        return;
      }
      current = current.next;
    }
  }
}
```

Časová složitost: $\mathcal{O}(n)$

Výhody spojového seznamu

Výhody spojového seznamu

- **Dynamická velikost** - roste a zmenšuje se podle potřeby

Výhody spojového seznamu

- **Dynamická velikost** - roste a zmenšuje se podle potřeby
- **Efektivní vkládání/mazání** na začátku spojového seznamu - $\mathcal{O}(1)$

Výhody spojového seznamu

- **Dynamická velikost** - roste a zmenšuje se podle potřeby
- **Efektivní vkládání/mazání** na začátku spojového seznamu - $\mathcal{O}(1)$
- **Žádná realokace** paměti při růstu

Výhody spojového seznamu

- **Dynamická velikost** - roste a zmenšuje se podle potřeby
- **Efektivní vkládání/mazání** na začátku spojového seznamu - $\mathcal{O}(1)$
- **Žádná realokace** paměti při růstu
- Vhodný pro **častou modifikaci** dat

Nevýhody spojového seznamu

Nevýhody spojového seznamu

- **Pomalejší přístup** k prvkům - složitost

Nevýhody spojového seznamu

- **Pomalejší přístup** k prvkům - složitost $\mathcal{O}(n)$ vs složitost u pole

Nevýhody spojového seznamu

- **Pomalejší přístup** k prvkům - složitost $\mathcal{O}(n)$ vs složitost u pole $\mathcal{O}(1)$

Nevýhody spojového seznamu

- **Pomalejší přístup** k prvkům - složitost $\mathcal{O}(n)$ vs složitost u pole $\mathcal{O}(1)$
- **Vyšší paměťová náročnost** - každý uzel ukládá odkaz

Nevýhody spojového seznamu

- **Pomalejší přístup** k prvkům - složitost $\mathcal{O}(n)$ vs složitost u pole $\mathcal{O}(1)$
- **Vyšší paměťová náročnost** - každý uzel ukládá odkaz
- **Nelze použít binární vyhledávání**

Nevýhody spojového seznamu

- **Pomalejší přístup** k prvkům - složitost $\mathcal{O}(n)$ vs složitost u pole $\mathcal{O}(1)$
- **Vyšší paměťová náročnost** - každý uzel ukládá odkaz
- **Nelze použít binární vyhledávání**
- Horší využití **cache** procesoru

Pole nebo spojový seznam?

Pole nebo spojový seznam?

Proč jsou pole v praxi často **rychlejší** než spojové seznamy, i když mají horší teoretickou složitost pro vkládání?

Pole nebo spojový seznam?

Proč jsou pole v praxi často **rychlejší** než spojové seznamy, i když mají horší teoretickou složitost pro vkládání?

Cache lokalita (cache locality)

Pole nebo spojový seznam?

Pole - prvky jsou v paměti **za sebou**

Pole nebo spojový seznam?

Pole - prvky jsou v paměti **za sebou**



Pole nebo spojový seznam?

Pole - prvky jsou v paměti **za sebou**



- Procesor načte celý blok do cache

Pole nebo spojový seznam?

Pole - prvky jsou v paměti **za sebou**



- Procesor načte celý blok do cache
- Přístup k dalším prvkům je **velmi rychlý**

Pole nebo spojový seznam?

Pole - prvky jsou v paměti **za sebou**



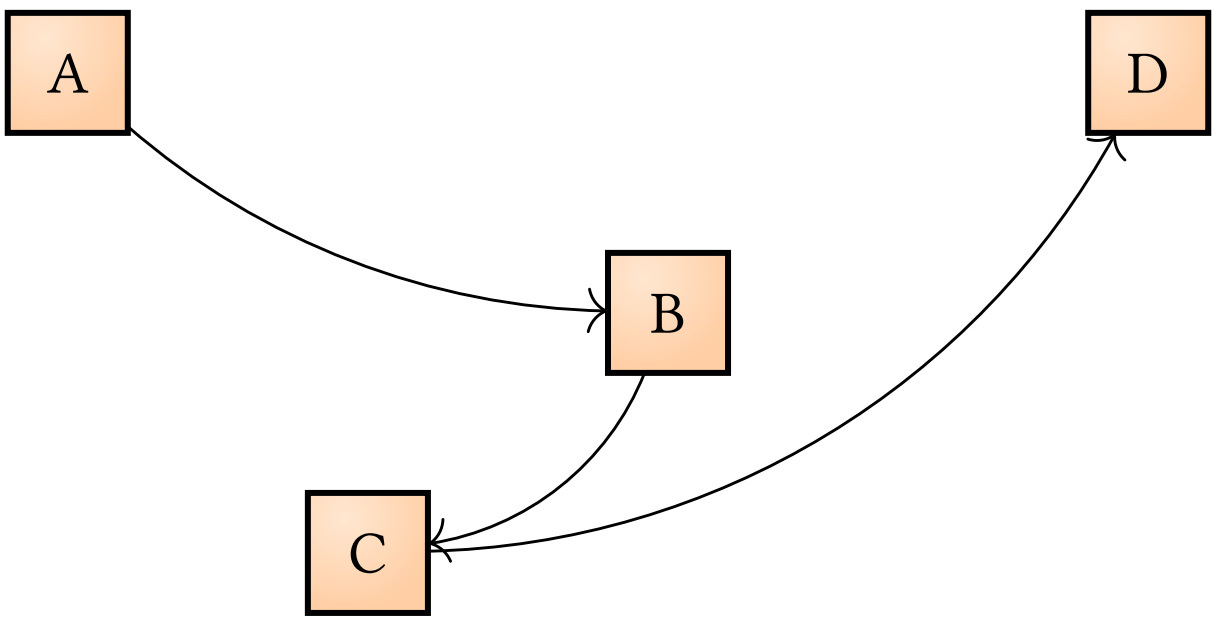
- Procesor načte celý blok do cache
- Přístup k dalším prvkům je **velmi rychlý**
- **Sekvenční přístup** je optimální

Pole nebo spojový seznam?

Spojový seznam - prvky jsou v paměti **roztroušené**

Pole nebo spojový seznam?

Spojový seznam - prvky jsou v paměti **roztroušené**



Pole nebo spojový seznam?

- Každý uzel může být **kdekoli v paměti**

Pole nebo spojový seznam?

- Každý uzel může být **kdekoli v paměti**
- **Cache miss** při každém skoku

Pole nebo spojový seznam?

- Každý uzel může být **kdekoli v paměti**
- **Cache miss** při každém skoku
- Pomalejší i pro sekvenční průchod

Pole nebo spojový seznam?

Srovnání časových složitostí

Operace	Pole	Spojový seznam
Přístup k i-tému prvku		

Pole nebo spojový seznam?

Srovnání časových složitostí

Operace	Pole	Spojový seznam
Přístup k i-tému prvku	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Vložení na začátek		

Pole nebo spojový seznam?

Srovnání časových složitostí

Operace	Pole	Spojový seznam
Přístup k i-tému prvku	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Vložení na začátek	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Vložení na konec		

Pole nebo spojový seznam?

Srovnání časových složitostí

Operace	Pole	Spojový seznam
Přístup k i-tému prvku	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Vložení na začátek	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Vložení na konec	$\mathcal{O}^*(1)$	$\mathcal{O}(n)$
Vyhledávání		

Pole nebo spojový seznam?

Srovnání časových složitostí

Operace	Pole	Spojový seznam
Přístup k i-tému prvku	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Vložení na začátek	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Vložení na konec	$\mathcal{O}^*(1)$	$\mathcal{O}(n)$
Vyhledávání	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Mazání prvku		

Pole nebo spojový seznam?

Srovnání časových složitostí

Operace	Pole	Spojový seznam
Přístup k i-tému prvku	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Vložení na začátek	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Vložení na konec	$\mathcal{O}^*(1)$	$\mathcal{O}(n)$
Vyhledávání	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Mazání prvku	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Pole nebo spojový seznam?

Poznámka 1: Vložení na konec pole má amortizovanou složitost $\mathcal{O}^*(1)$ (při dynamickém poli, které se realokuje na dvojnásobnou velikost)

Pole nebo spojový seznam?

Poznámka 1: Vložení na konec pole má amortizovanou složitost $\mathcal{O}^*(1)$ (při dynamickém poli, které se realokuje na dvojnásobnou velikost)

Poznámka 2: Přidání na konec spojového seznamu může mít složitost $\mathcal{O}(1)$, pokud je implementován s dodatečným ukazatelem na poslední prvek

Praxe

Praxe

- Pole jsou **rychlejší** pro většinu operací

Praxe

- Pole jsou **rychlejší** pro většinu operací
- Cache efekt > teoretická složitost

Praxe

- Pole jsou **rychlejší** pro většinu operací
- Cache efekt > teoretická složitost
- Moderní procesory mají silný **prefetch** pro sekvenční přístup

Praxe

- Pole jsou **rychlejší** pro většinu operací
- Cache efekt > teoretická složitost
- Moderní procesory mají silný **prefetch** pro sekvenční přístup
- Spojový seznam použijte jen když **opravdu** potřebujete

Praxe

- Pole jsou **rychlejší** pro většinu operací
- Cache efekt > teoretická složitost
- Moderní procesory mají silný **prefetch** pro sekvenční přístup
- Spojový seznam použijte jen když **opravdu** potřebujete
  - Časté vkládání/mazání na začátku
  - Nelze odhadnout velikost

Praxe

- Pole jsou **rychlejší** pro většinu operací
- Cache efekt > teoretická složitost
- Moderní procesory mají silný **prefetch** pro sekvenční přístup
- Spojový seznam použijte jen když **opravdu** potřebujete
  - Časté vkládání/mazání na začátku
  - Nelze odhadnout velikost

Nicméně spojový seznam a jeho princip je **použit jako konstrukční blok složitějších struktur**, které mají i v praxi větší výhody než pole

Binární strom (Binary Tree)

Binární strom

Binární strom je

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**
- Rozlišujeme

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**
- Rozlišujeme **levého potomka** a

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**
- Rozlišujeme **levého potomka** a **pravého potomka**
- Existuje právě jeden

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**
- Rozlišujeme **levého potomka** a **pravého potomka**
- Existuje právě jeden **kořen** stromu

Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**
- Rozlišujeme **levého potomka** a **pravého potomka**
- Existuje právě jeden **kořen** stromu

Uzel bez potomků se nazývá

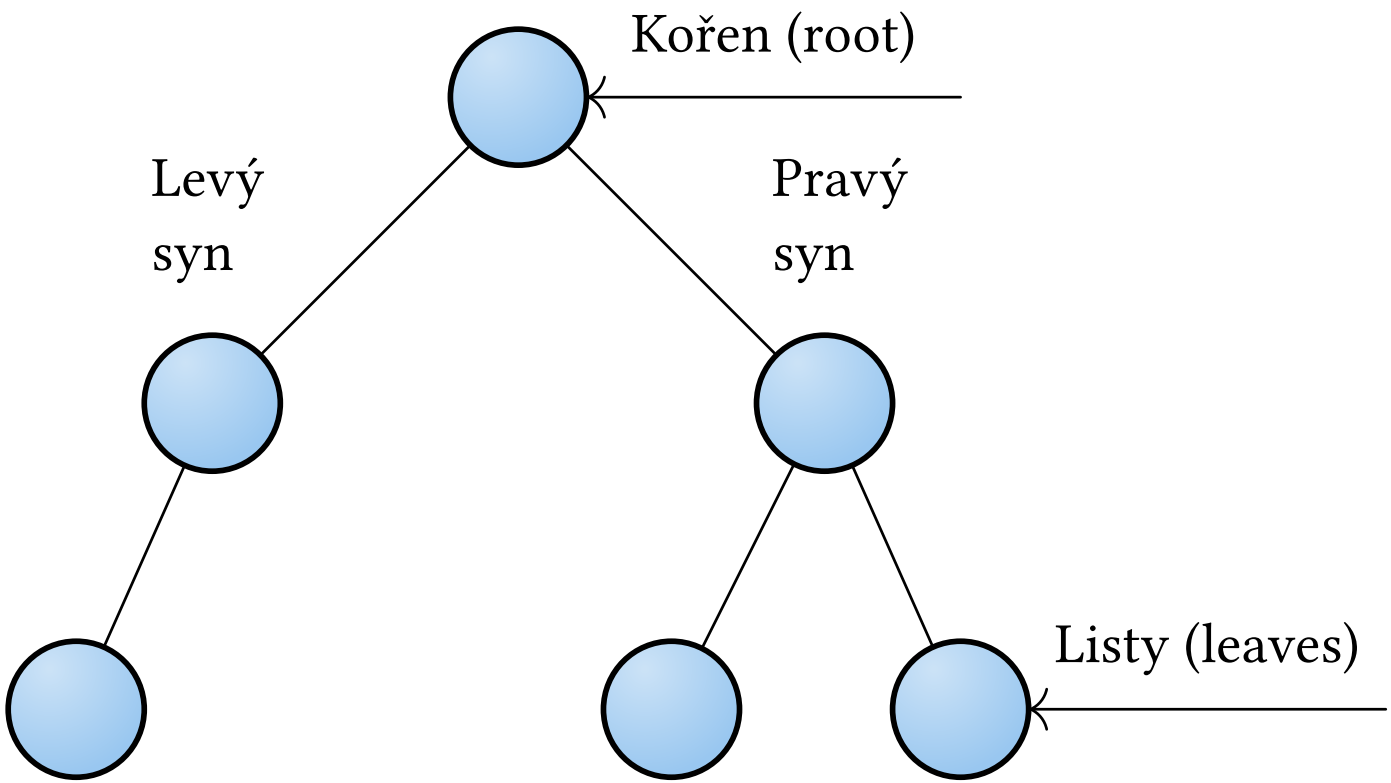
Binární strom

Binární strom je rekurzivní datová struktura s následujícími vlastnostmi:

- Každý uzel má **maximálně 2 potomky**
- Rozlišujeme **levého potomka** a **pravého potomka**
- Existuje právě jeden **kořen** stromu

Uzel bez potomků se nazývá **list**

Vizualizace



Implementace

Implementace

```
class TreeNode<T> {
  value: T;
  left: TreeNode<T> | null = null;
  right: TreeNode<T> | null = null;

  constructor(value: T) {
    this.value = value;
  }
}
```

Implementace

```
class BinaryTree<T> {
  protected root: TreeNode<T> | null = null;
}
```

Typy binárních stromů

Plný binární strom

(Full)

Typy binárních stromů

Plný binární strom

(Full)

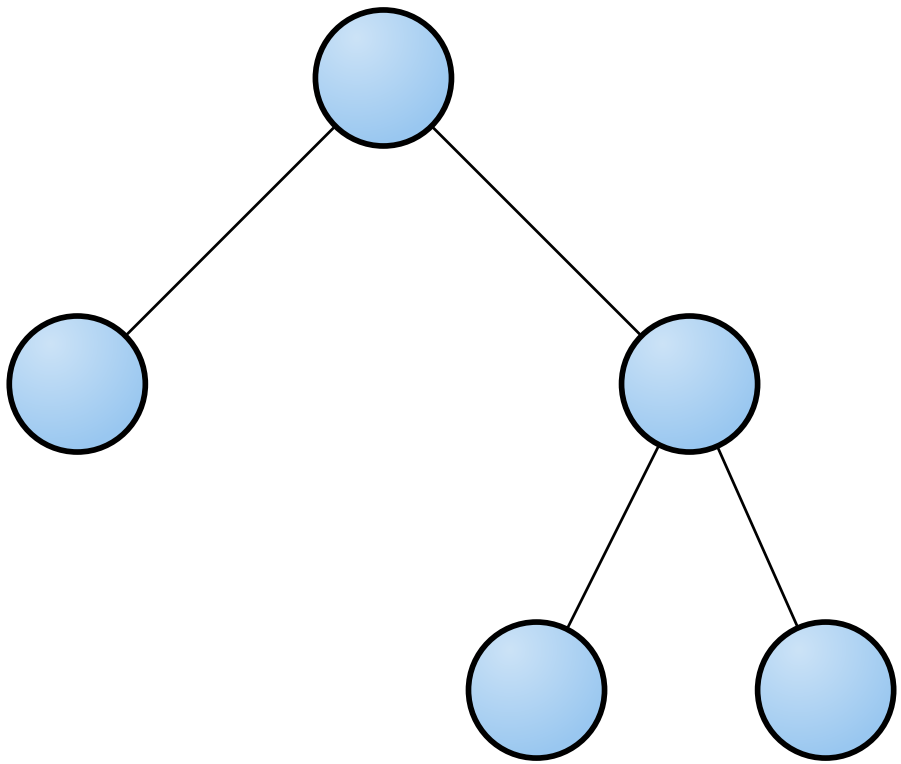
- Každý uzel má 0 nebo 2 děti

Typy binárních stromů

Plný binární strom

(Full)

- Každý uzel má 0 nebo 2 děti



Typy binárních stromů

Kompletní binární strom

(Complete)

Typy binárních stromů

Kompletní binární strom

(Complete)

- Všechny úrovně plné kromě poslední

Typy binárních stromů

Kompletní binární strom

(Complete)

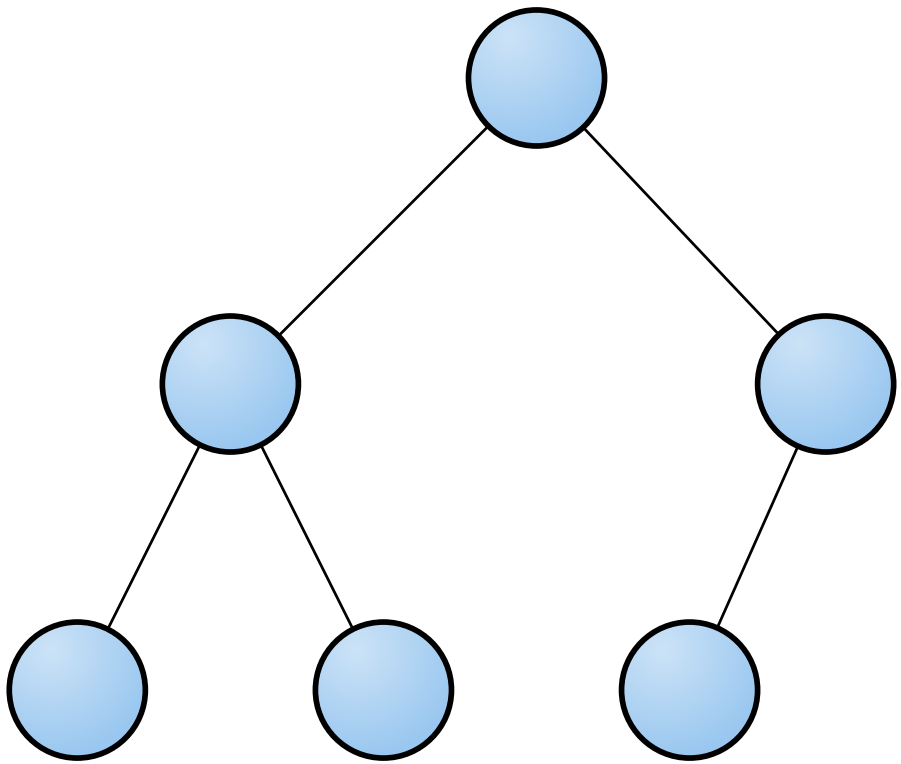
- Všechny úrovně plné kromě poslední
- Poslední úroveň zaplněná zleva doprava

Typy binárních stromů

Kompletní binární strom

(Complete)

- Všechny úrovně plné kromě poslední
- Poslední úroveň zaplněná zleva doprava



Typy binárních stromů

Perfektní binární strom

(Perfect)

Typy binárních stromů

Perfektní binární strom

(Perfect)

- Všechny listy na stejné úrovni

Typy binárních stromů

Perfektní binární strom

(Perfect)

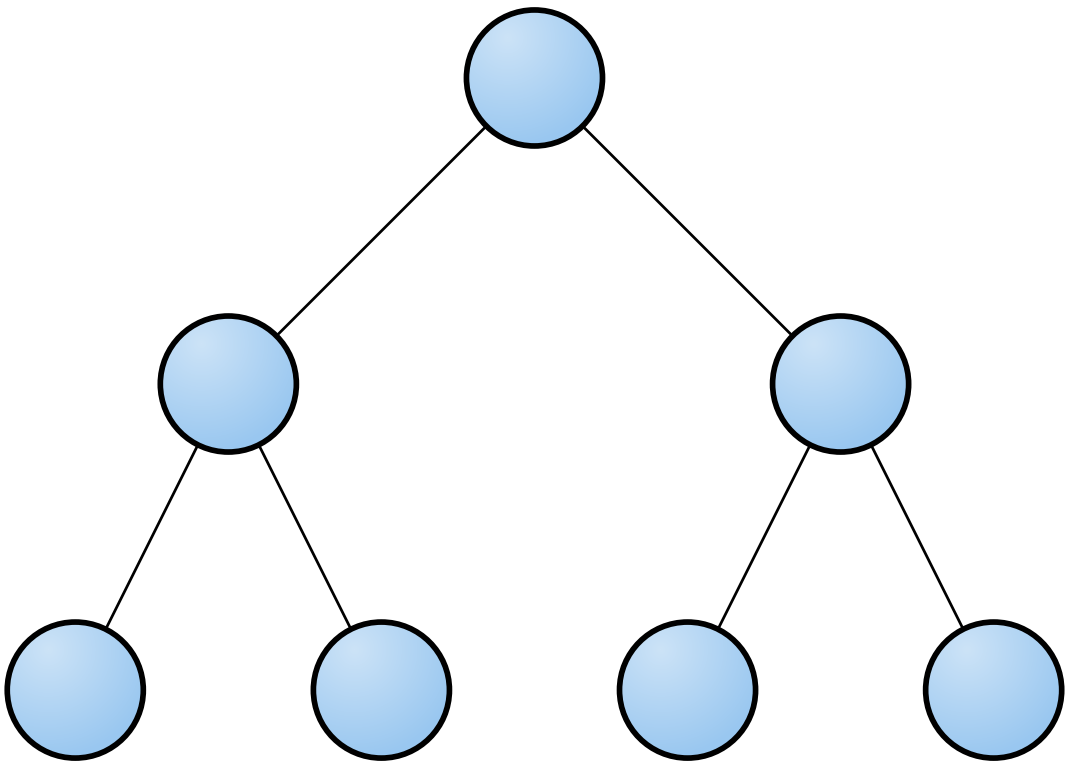
- Všechny listy na stejné úrovni
- Počet uzlů: $2^h - 1$
  (kde h = výška)

Typy binárních stromů

Perfektní binární strom

(Perfect)

- Všechny listy na stejné úrovni
- Počet uzlů: $2^h - 1$
  (kde h = výška)



Procházení binárních stromů

Pre-order

Pre-order průchod =

Pre-order

Pre-order průchod = **Aktuální, Levý, Pravý**

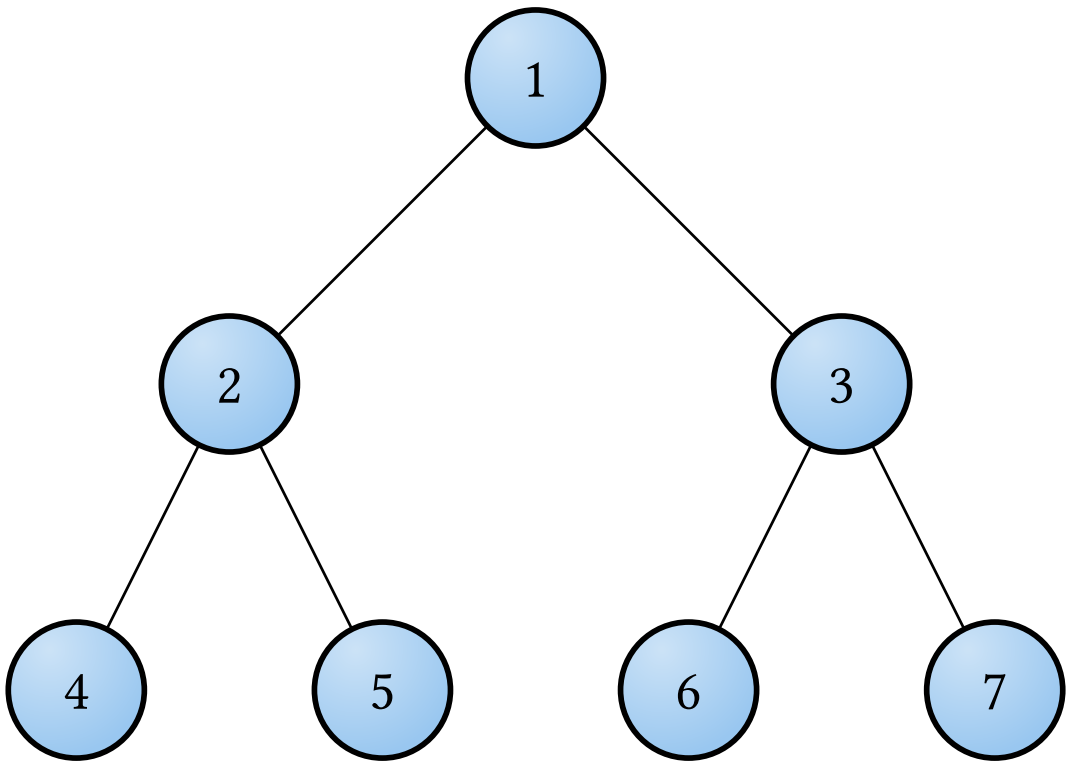
Pre-order

Pre-order průchod = **Aktuální, Levý, Pravý**

```
preOrder(node: TreeNode<T> | null): void {
  if (!node) return;

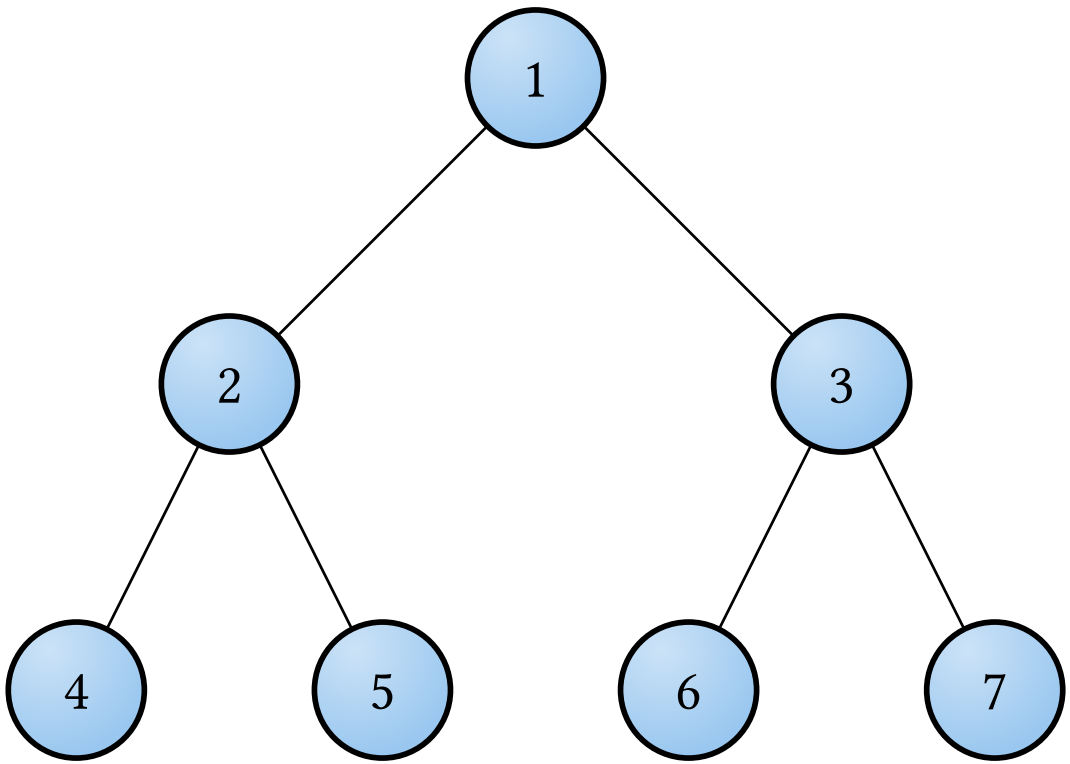
  console.log(node.value);    // Process current node
  this.preOrder(node.left);   // Process left subtree
  this.preOrder(node.right);  // Process right subtree
}
```

Pre-order



Výstup:

Pre-order



Výstup: 1, 2, 4, 5, 3, 6, 7

In-order

In-order průchod =

In-order

In-order průchod = **Levý, Aktuální, Pravý**

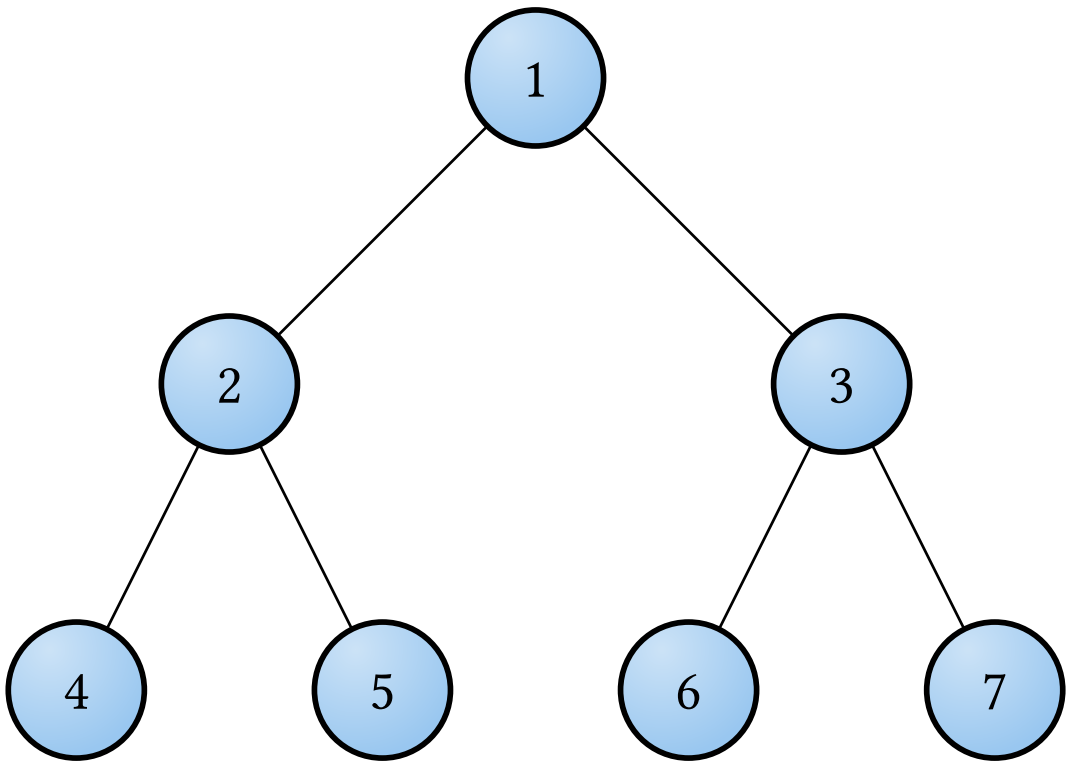
In-order

In-order průchod = **Levý, Aktuální, Pravý**

```
inOrder(node: TreeNode<T> | null): void {
  if (!node) return;

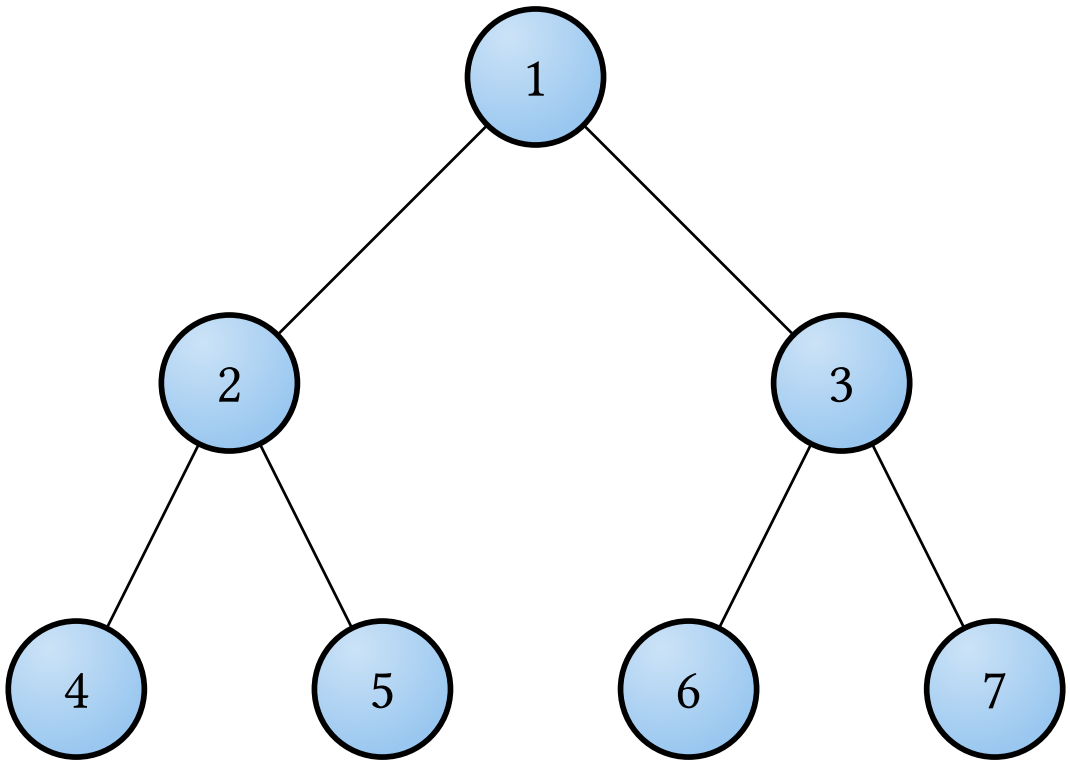
  this.inOrder(node.left);     // Left subtree
  console.log(node.value);     // Process root
  this.inOrder(node.right);    // Right subtree
}
```

In-order



Výstup:

In-order



Výstup: 4, 2, 5, 1, 6, 3, 7

Post-order

Post-order průchod =

Post-order

Post-order průchod = **Levý, Pravý, Aktuální**

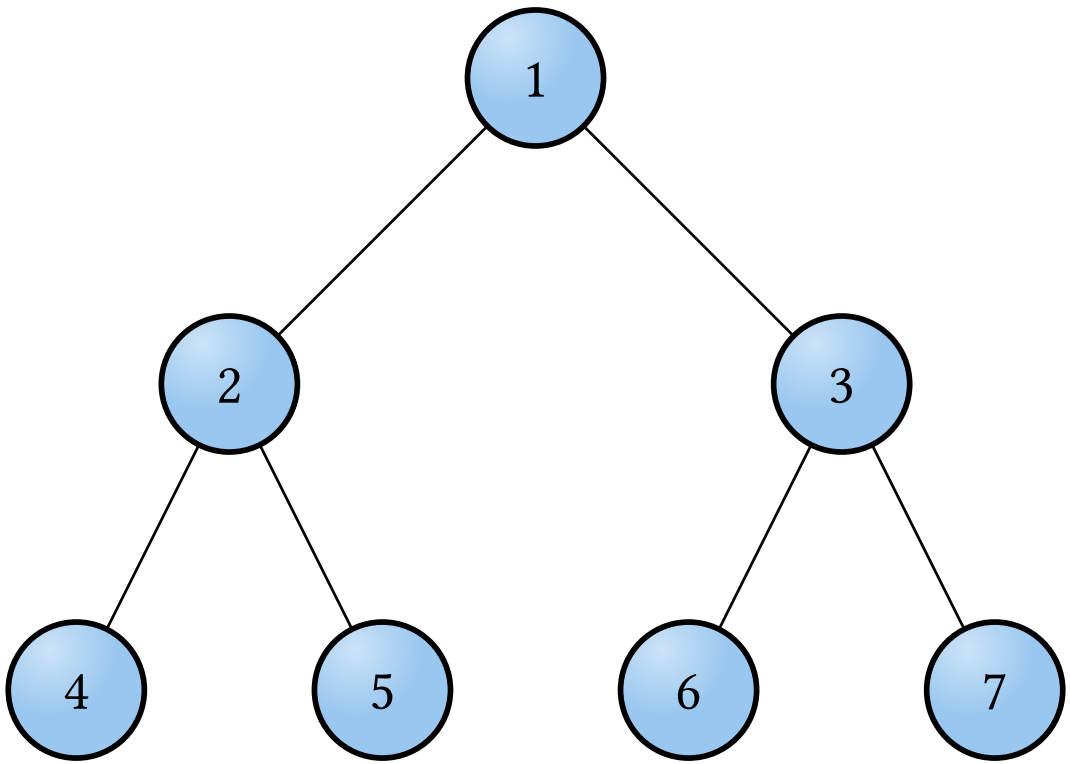
Post-order

Post-order průchod = **Levý, Pravý, Aktuální**

```
postOrder(node: TreeNode<T> | null): void {
  if (!node) return;

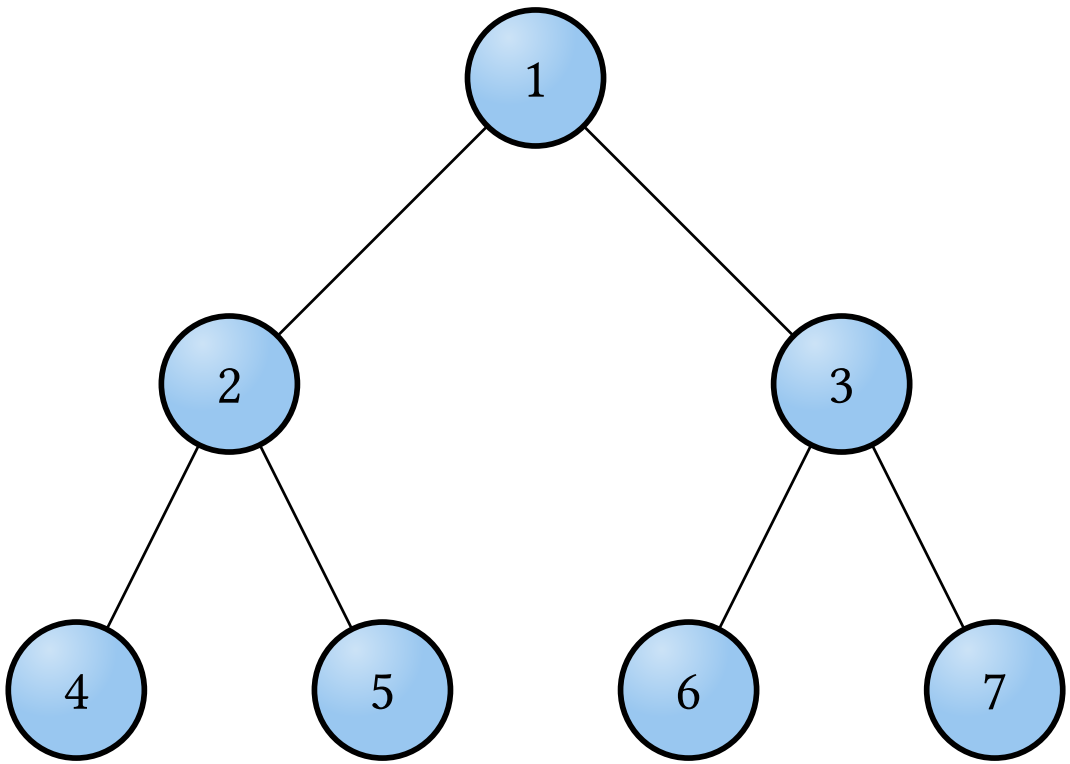
  this.postOrder(node.left);   // Left subtree
  this.postOrder(node.right);  // Right subtree
  console.log(node.value);     // Process root
}
```

Post-order



Výstup:

Post-order



Výstup: 4, 5, 2, 6, 7, 3, 1

Iterativní průchody stromu

Průchody stromu lze implementovat **bez**

Iterativní průchody stromu

Průchody stromu lze implementovat **bez rekurze** pomocí

Iterativní průchody stromu

Průchody stromu lze implementovat **bez rekurze** pomocí explicitního **zásobníku**

Iterativní průchody stromu

Průchody stromu lze implementovat **bez rekurze** pomocí explicitního **zásobníku**

Výhody iterativního přístupu:

Iterativní průchody stromu

Průchody stromu lze implementovat **bez rekurze** pomocí explicitního **zásobníku**

Výhody iterativního přístupu:

- Lepší kontrola nad pamětí

Iterativní průchody stromu

Průchody stromu lze implementovat **bez rekurze** pomocí explicitního **zásobníku**

Výhody iterativního přístupu:

- Lepší kontrola nad pamětí
- Vyhneme se stack overflow u hlubokých stromů

Iterativní průchody stromu

Průchody stromu lze implementovat **bez rekurze** pomocí explicitního **zásobníku**

Výhody iterativního přístupu:

- Lepší kontrola nad pamětí
- Vyhneme se stack overflow u hlubokých stromů
- Praktické použití zásobníku

Iterativní průchody stromu

Pre-order iterativně

Iterativní průchody stromu

Pre-order iterativně

```
preOrderIterative(node: TreeNode<T> | null): void {
  if (!node) return;

  const stack = new Stack<TreeNode<T>>();
  stack.push(node);
```

Iterativní průchody stromu

```
  while (!stack.isEmpty()) {
    const current = stack.pop();
    if (!current) continue;

    console.log(current.value);  // Process current

    // Push right first, then left (LIFO)
    if (current.right) stack.push(current.right);
    if (current.left) stack.push(current.left);
  }
}
```

Iterativní průchody stromu

In-order iterativně

Iterativní průchody stromu

In-order iterativně

```
inOrderIterative(node: TreeNode<T> | null): void {
  const stack = new Stack<TreeNode<T>>();
  let current = node;

  while (current || !stack.isEmpty()) {
    // Go to leftmost node
    while (current) {
      stack.push(current);
      current = current.left;
    }
```

Iterativní průchody stromu

```
    // Process node
    current = stack.pop();
    if (current) {
      console.log(current.value);
      current = current.right;  // Move to right subtree
    }
  }
}
```

Iterativní průchody stromu

Post-order iterativně

Iterativní průchody stromu

Post-order iterativně

```
postOrderIterative(node: TreeNode<T> | null): void {
  if (!node) return;

  const stack1 = new Stack<TreeNode<T>>();
  const stack2 = new Stack<TreeNode<T>>();

  stack1.push(node);
```

Iterativní průchody stromu

```
  while (!stack1.isEmpty()) {
    const current = stack1.pop();
    if (!current) continue;

    stack2.push(current);

    if (current.left) stack1.push(current.left);
    if (current.right) stack1.push(current.right);
  }
```

Iterativní průchody stromu

```
  // Print in reverse order
  while (!stack2.isEmpty()) {
    const current = stack2.pop();
    if (current) console.log(current.value);
  }
}
```

Binární vyhledávací strom (BST)

Binární vyhledávací strom

BST (Binary Search Tree) je

Binární vyhledávací strom

BST (Binary Search Tree) je binární strom s vlastnostmi:

Binární vyhledávací strom

BST (Binary Search Tree) je binární strom s vlastnostmi:

Pro každý uzel platí:

Binární vyhledávací strom

BST (Binary Search Tree) je binární strom s vlastnostmi:

Pro každý uzel platí:

- Všechny hodnoty v **levém podstromu** jsou **menší**

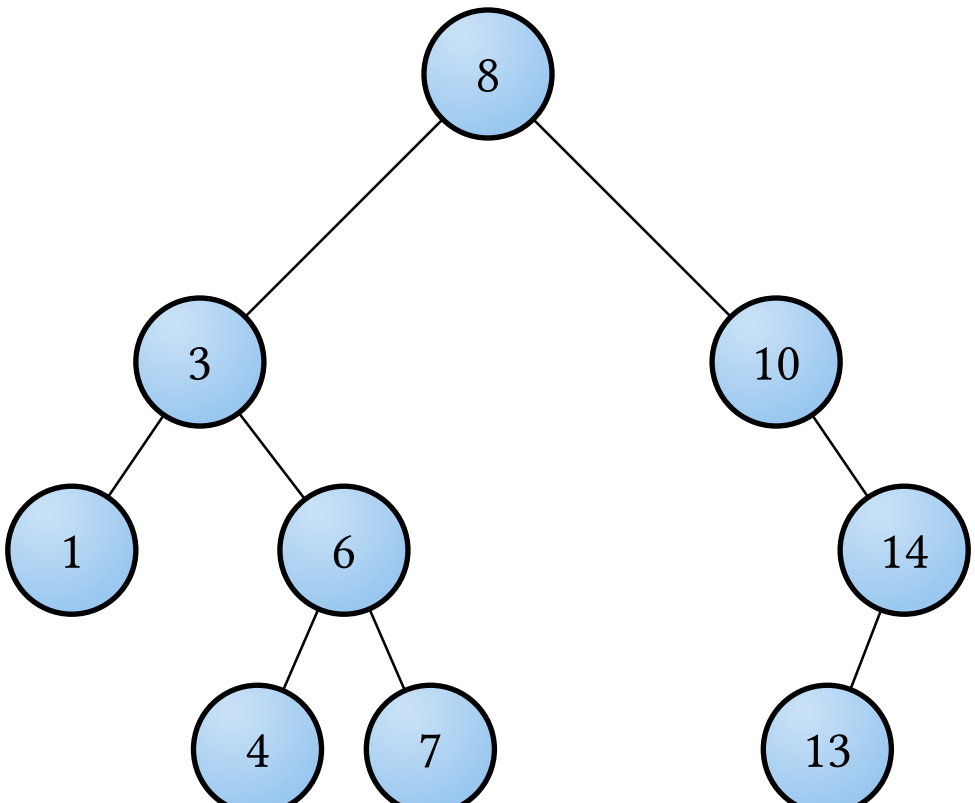
Binární vyhledávací strom

BST (Binary Search Tree) je binární strom s vlastnostmi:

Pro každý uzel platí:

- Všechny hodnoty v **levém podstromu** jsou **menší**
- Všechny hodnoty v **pravém podstromu** jsou **větší**

Vizualizace BST



Vyhledávání v BST

Vyhledávání v BST

```
search(
  value: T,
  node: TreeNode<T> | null = this.root
): TreeNode<T> | null {
  // Node not found
  if (!node) return null;

  // Found it!
  if (value === node.value) return node;
```

Vyhledávání v BST

```
  // Search in left subtree
  if (value < node.value) {
    return this.search(value, node.left);
  }

  // Search in right subtree
  return this.search(value, node.right);
}
```

Vložení do BST

Vložení do BST

```
insert(value: T): void {
  const newNode = new TreeNode(value);

  if (!this.root) {
    this.root = newNode;
    return;
  }

  this.insertNode(this.root, newNode);
}
```

Vložení do BST

```
private insertNode(
  node: TreeNode<T>, newNode: TreeNode<T>
): void {
  if (newNode.value < node.value) {
    if (!node.left) node.left = newNode;
    else this.insertNode(node.left, newNode);
  } else {
    if (!node.right) node.right = newNode;
    else this.insertNode(node.right, newNode);
  }
}
```

Odstranění z BST

Odstranění z BST

Tři případy:

Odstranění z BST

Tři případy:

1. **Uzel je list** - jednoduché odstranění

Odstranění z BST

Tři případy:

1. **Uzel je list** - jednoduché odstranění
2. **Uzel má jednoho potomka** - nahraď uzlem tímto potomkem

Odstranění z BST

Tři případy:

1. **Uzel je list** - jednoduché odstranění
2. **Uzel má jednoho potomka** - nahraď uzlem tímto potomkem
3. **Uzel má dva potomky** - najdi následníka (nejmenší v pravém podstromu) a nahraď aktuální uzel tímto následníkem

Odstranění z BST

```
remove(value: T): void {
  this.root = this.removeNode(this.root, value);
}
```

Odstranění z BST

```
private removeNode(
  node: TreeNode<T> | null, value: T
): TreeNode<T> | null {
  if (!node) return null;

  if (value < node.value) { // Search in left subtree
    node.left = this.removeNode(node.left, value);
  } else if (value > node.value) { // Search in right subtree
    node.right = this.removeNode(node.right, value);
  } else {
    // Found the node to remove
```

Odstranění z BST

```
// Case 1: Leaf node
if (!node.left && !node.right) return null;

// Case 2: One child
if (!node.left) return node.right;
if (!node.right) return node.left;
```

Odstranění z BST

```
    // Case 3: Two children
    const minRight = this.findMin(node.right);
    node.value = minRight.value;
    node.right = this.removeNode(node.right, minRight.value);
  }

  return node;
}
```

Odstranění z BST

```
private findMin(
  node: TreeNode<T>
): TreeNode<T> {
  if (!node.left) return node;
  return this.findMin(node.left);
}
```

Rozsah složitostí

Pokud máme algoritmus s lineární složitostí $\mathcal{O}(n)$, tak to znamená, že

Rozsah složitostí

Pokud máme algoritmus s lineární složitostí $\mathcal{O}(n)$, tak to znamená, že počet operací roste lineárně s velikostí vstupu/velikostí dat

Rozsah složitostí

Pokud máme algoritmus s lineární složitostí $\mathcal{O}(n)$, tak to znamená, že počet operací roste lineárně s velikostí vstupu/velikostí dat

- $\mathcal{O}(n)$ - pro milion prvků očekáváme přibližně miliony operací ($a * n$, kde a je konstanta)
- $\mathcal{O}(\log n)$ - pro milion prvků očekáváme přibližně 20 operací ($a * \log(n)$, kde a je konstanta)
  - Zápisem $\log n$ se myslí logaritmus o základu 2

Výhody binárního vyhledávacího stromu

Výhody binárního vyhledávacího stromu

- **Rychlé vyhledávání** - složitost

Výhody binárního vyhledávacího stromu

- **Rychlé vyhledávání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**

Výhody binárního vyhledávacího stromu

- **Rychlé vyhledávání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**
- **Rychlé vkládání a mazání** - složitost

Výhody binárního vyhledávacího stromu

- **Rychlé vyhledávání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**
- **Rychlé vkládání a mazání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**

Výhody binárního vyhledávacího stromu

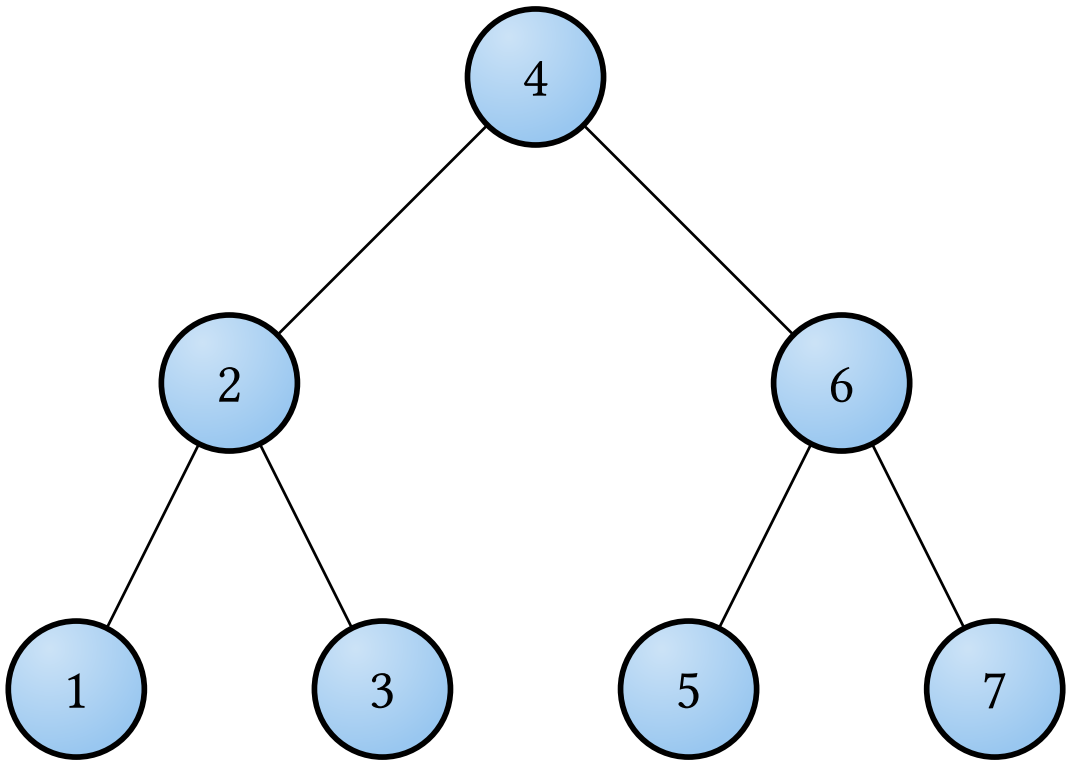
- **Rychlé vyhledávání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**
- **Rychlé vkládání a mazání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**
- **Udržuje pořadí** - in-order dává seřazené prvky

Výhody binárního vyhledávacího stromu

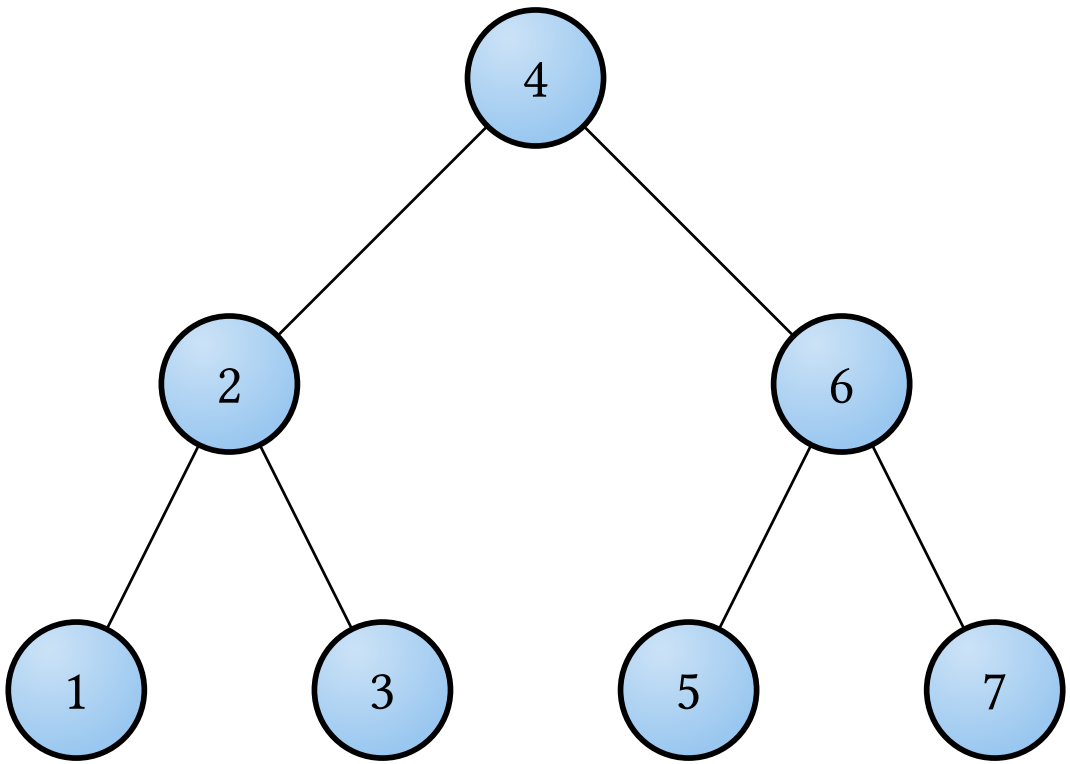
- **Rychlé vyhledávání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**
- **Rychlé vkládání a mazání** - složitost $\mathcal{O}^*(\log n)$ **v průměru**
- **Udržuje pořadí** - in-order dává seřazené prvky
- Dynamická struktura - roste podle potřeby

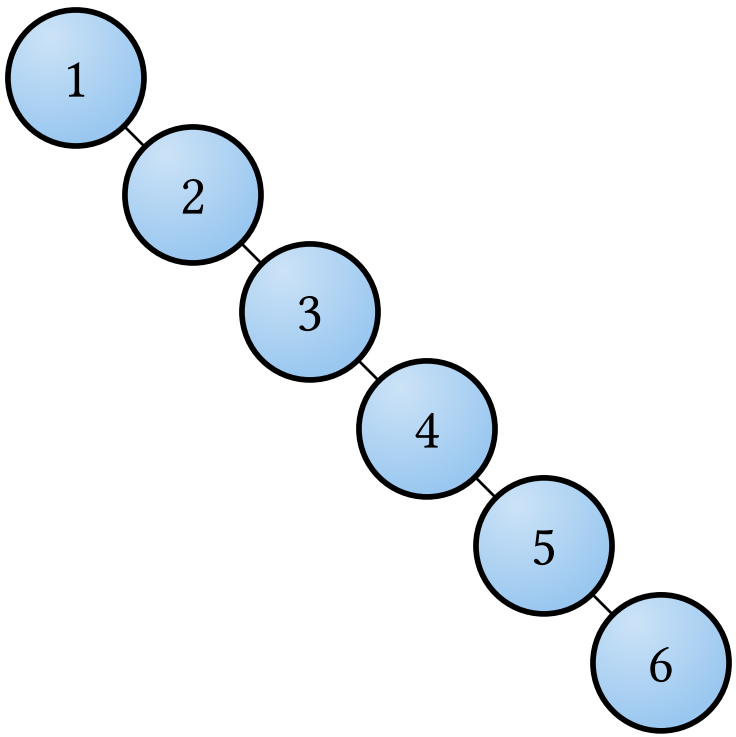
Problém vyváženosti

Problém vyváženosti



Problém vyváženosti





Problém vyváženosti

Nevyvážený binární strom může **degenerovat** na

Problém vyváženosti

Nevyvážený binární strom může **degenerovat** na spojový seznam

Problém vyváženosti

Nevyvážený binární strom může **degenerovat** na spojový seznam

V tomto případě budou operace vyhledávání, vkládání a mazání mít složitost

Problém vyváženosti

Nevyvážený binární strom může **degenerovat** na spojový seznam

V tomto případě budou operace vyhledávání, vkládání a mazání mít složitost $\mathcal{O}(n)$

Problém vyváženosti

Nevyvážený binární strom může **degenerovat** na spojový seznam

V tomto případě budou operace vyhledávání, vkládání a mazání mít složitost $\mathcal{O}(n)$

Pokud bychom dokázali strom udržovat vyvážený, tak by operace s ním měly stálou složitost $\mathcal{O}(\log n)$

Balancování binárních vyhledávacích stromů

Výška stromu

Výška stromu je

Výška stromu

Výška stromu je maximální počet hran na nejdelší cestě z kořene do listu

Výška stromu

Výška stromu je maximální počet hran na nejdelší cestě z kořene do listu

Výška podstromu je

Výška stromu

Výška stromu je maximální počet hran na nejdelší cestě z kořene do listu

Výška podstromu je maximální počet hran v podstromu začínajícím v daném uzlu do listu

Výška stromu

Výška stromu je maximální počet hran na nejdelší cestě z kořene do listu

Výška podstromu je maximální počet hran v podstromu začínajícím v daném uzlu do listu

Výšku podstromu u označíme $h(u)$

Výška stromu

Balanční faktor (δ) uzlu u:

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva
- 2, pokud je

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva
- 2, pokud je uzel

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva
- 2, pokud je uzel „moc převážený“ doprava

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva
- 2, pokud je uzel „moc převážený“ doprava
- -2, pokud je

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva
- 2, pokud je uzel „moc převážený“ doprava
- -2, pokud je uzel

Výška stromu

Balanční faktor (δ) uzlu u:

$$\delta(u) = h(\text{u.right}) - h(\text{u.left})$$

Hodnoty balančního faktoru mohou být

- 0, pokud je uzel vyvážený
- 1, pokud je uzel „lehce převážený“ doprava
- -1, pokud je uzel „lehce převážený“ doleva
- 2, pokud je uzel „moc převážený“ doprava
- -2, pokud je uzel „moc převážený“ doleva

AVL strom

AVL strom - první samobalancující BST (1962)

AVL strom

AVL strom - první samobalancující BST (1962)

Pojmenován po autorech: **A**delson-**V**elsky a **L**andis

AVL strom

AVL strom - první samobalancující BST (1962)

Pojmenován po autorech: **A**delson-**V**elsky a **L**andis

Pravidlo:

AVL strom

AVL strom - první samobalancující BST (1962)

Pojmenován po autorech: **A**delson-**V**elsky a **L**andis

Pravidlo:

Pro každý uzel u platí: $|\delta(u)| \leq 1$

AVL strom

AVL strom - první samobalancující BST (1962)

Pojmenován po autorech: **A**delson-**V**elsky a **L**andis

Pravidlo:

Pro každý uzel u platí: $|\delta(u)| \leq 1$

Po každé operaci se strom **automaticky vyváží** (opraví) pomocí takzvaných

AVL strom

AVL strom - první samobalancující BST (1962)

Pojmenován po autorech: **A**delson-**V**elsky a **L**andis

Pravidlo:

Pro každý uzel u platí: $|\delta(u)| \leq 1$

Po každé operaci se strom **automaticky vyváží** (opraví) pomocí takzvaných **rotací**

Pravá rotace

Každá rotace má své podmínky, dle kterých se pozná, jaká se má provést

Pravá rotace

Každá rotace má své podmínky, dle kterých se pozná, jaká se má provést

Pravá rotace se provádí na uzlu u, pokud platí

Pravá rotace

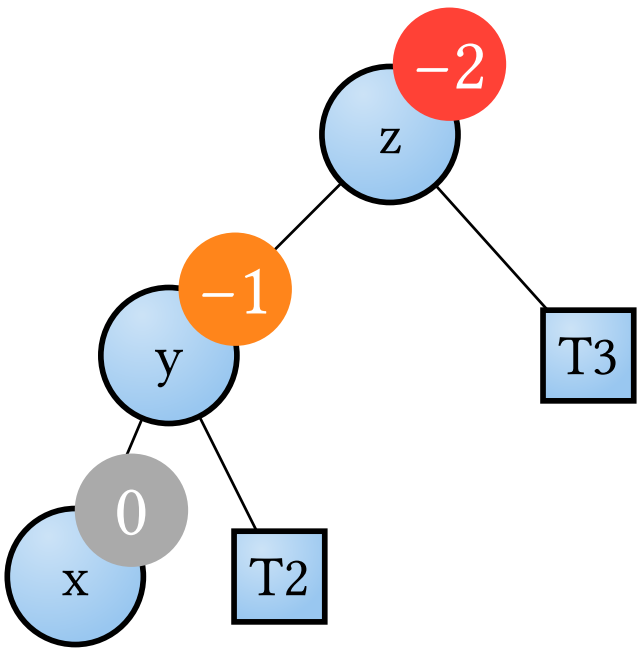
Každá rotace má své podmínky, dle kterých se pozná, jaká se má provést

Pravá rotace se provádí na uzlu u, pokud platí

$$\delta(u) = -2 \land \delta(\text{u.left}) = -1$$

Pravá rotace

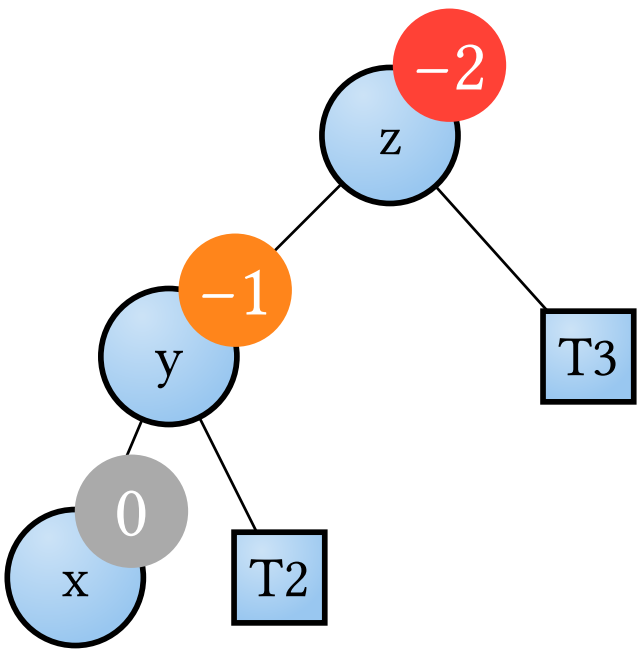
Před rotací



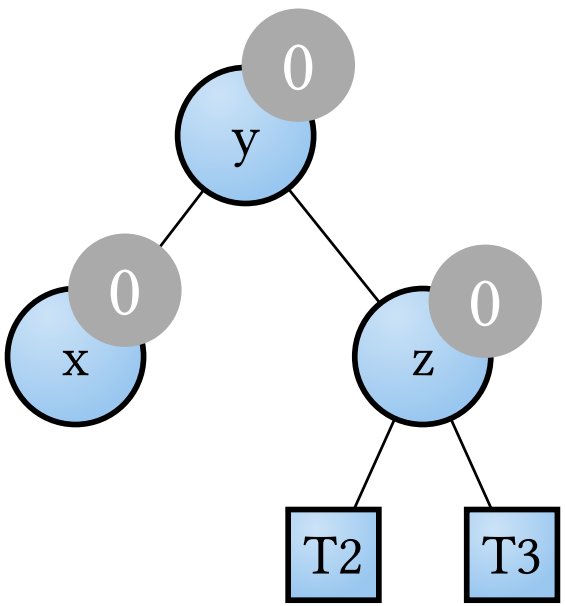
Po rotaci

Pravá rotace

Před rotací



Po rotaci



Levá rotace

Levá rotace se provádí na uzlu u, pokud platí

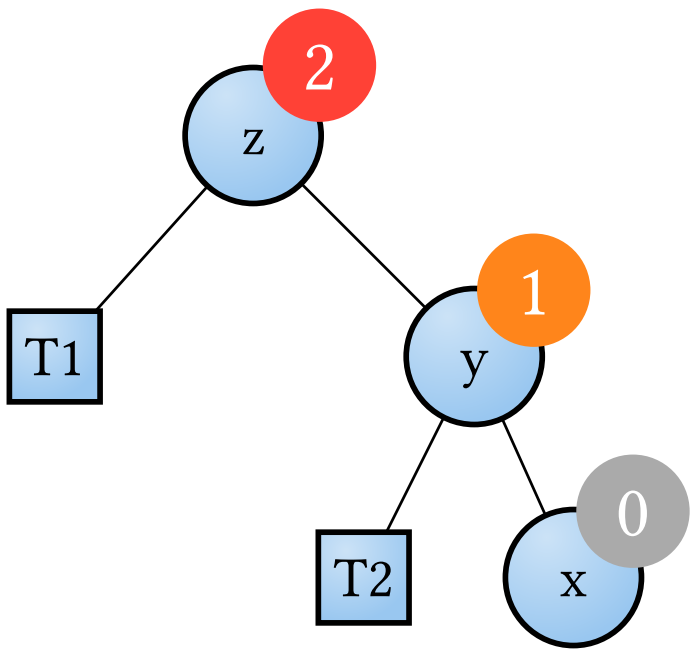
Levá rotace

Levá rotace se provádí na uzlu u, pokud platí

$$\delta(u) = 2 \wedge \delta(\text{u.right}) = 1$$

Levá rotace

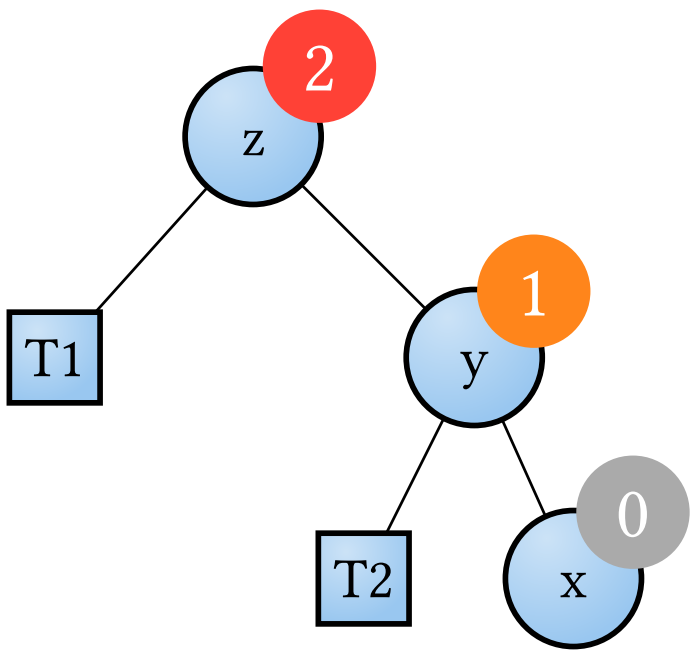
Před rotací



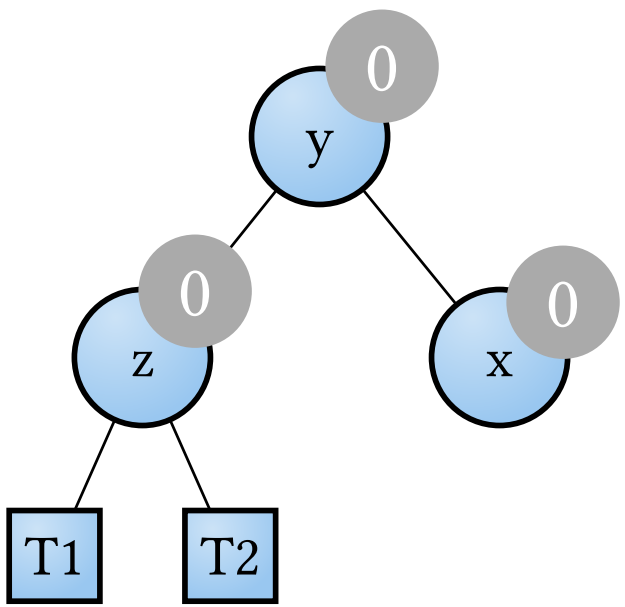
Po rotaci

Levá rotace

Před rotací



Po rotaci



Levo-pravá rotace

Levo-pravá rotace se provádí na uzlu u, pokud platí

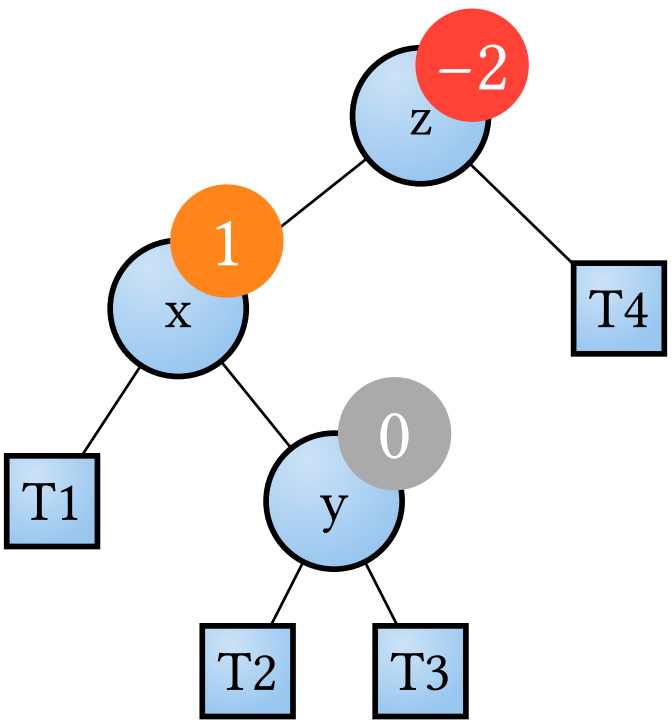
Levo-pravá rotace

Levo-pravá rotace se provádí na uzlu u, pokud platí

$$\delta(u) = -2 \wedge \delta(\mathrm{u.left}) = 1$$

Levo-pravá rotace

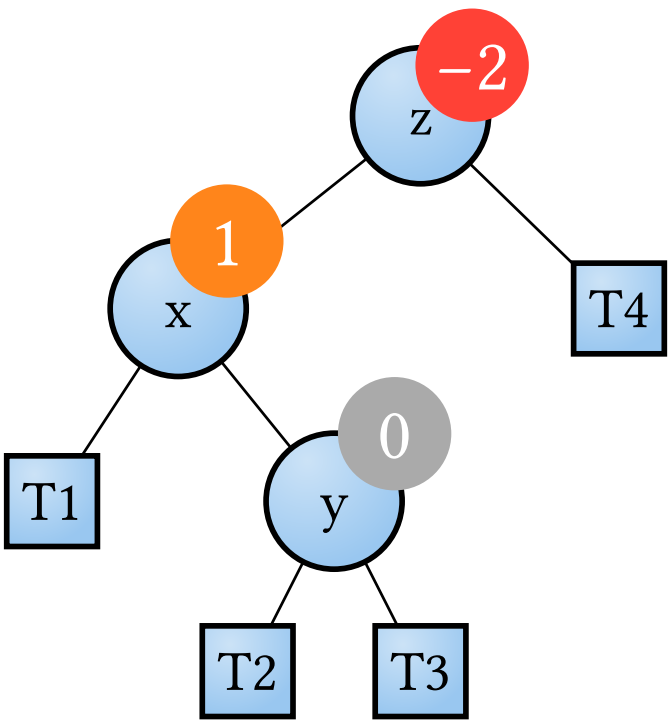
Před rotací



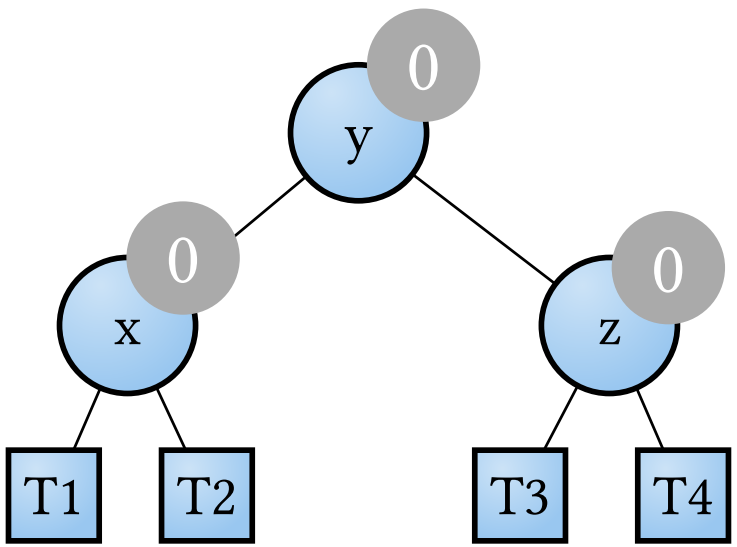
Po rotaci

Levo-pravá rotace

Před rotací



Po rotaci



Pravo-levá rotace

Pravo-levá rotace se provádí na uzlu u, pokud platí

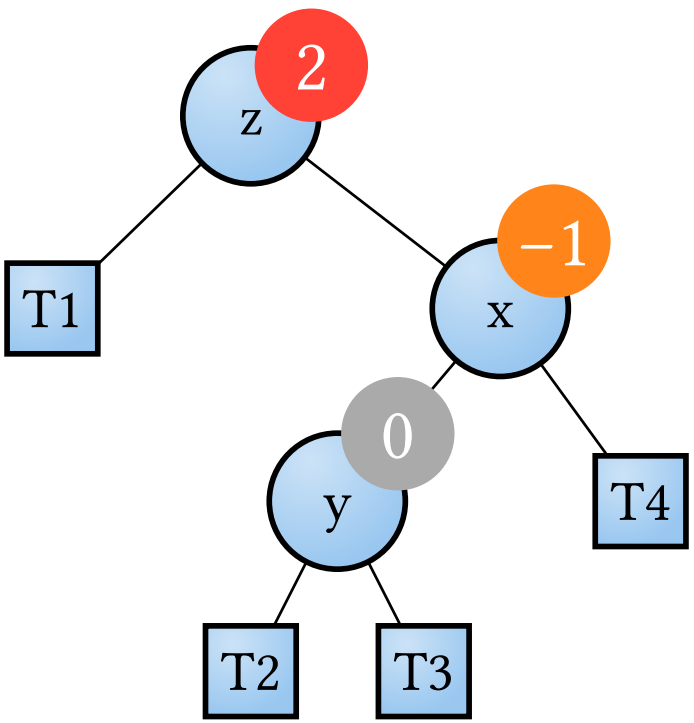
Pravo-levá rotace

Pravo-levá rotace se provádí na uzlu u, pokud platí

$$\delta(u) = 2 \wedge \delta(\mathrm{u.right}) = -1$$

Pravo-levá rotace

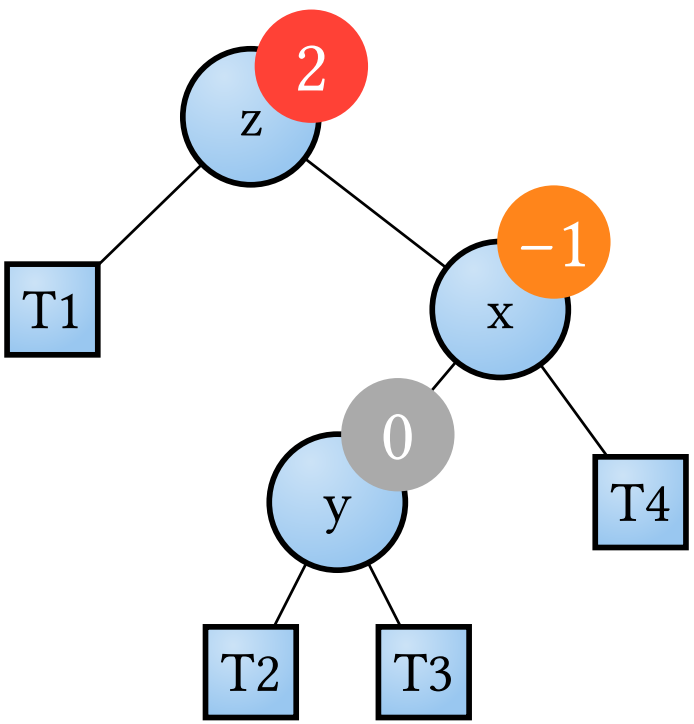
Před rotací



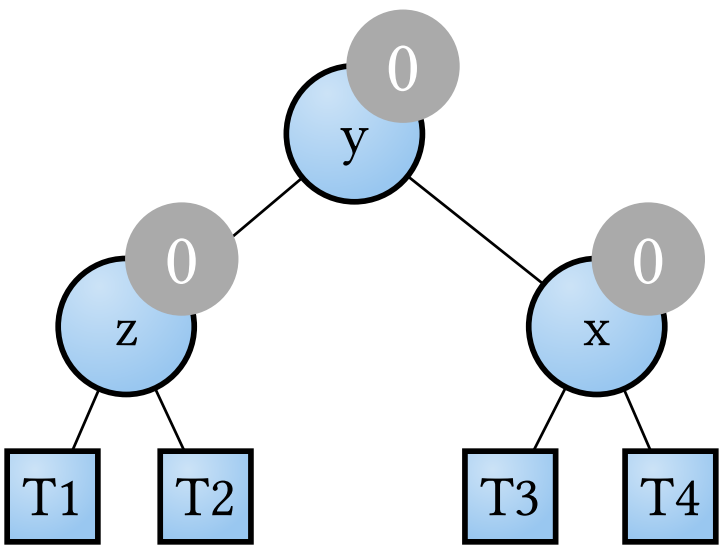
Po rotaci

Pravo-levá rotace

Před rotací



Po rotaci



Časová složitost rotací

Časová složitost rotací je

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Kolik rotací je nutné provést při vkládání/mazání?

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Kolik rotací je nutné provést při vkládání/mazání?

- Při vkládání:

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Kolik rotací je nutné provést při vkládání/mazání?

- Při vkládání: $\mathcal{O}(\log n)$
- Při mazání:

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Kolik rotací je nutné provést při vkládání/mazání?

- Při vkládání: $\mathcal{O}(\log n)$
- Při mazání: $\mathcal{O}(\log n)$

Proč?

Časová složitost rotací

Časová složitost rotací je $\mathcal{O}(1)$

Kolik rotací je nutné provést při vkládání/mazání?

- Při vkládání: $\mathcal{O}(\log n)$
- Při mazání: $\mathcal{O}(\log n)$

Proč?

Protože při vkládání/mazání se může stát, že se strom dokáže korektně vyvážit až na kořeni

Časová složitost rotací

Jelikož je hloubka stromu $\mathcal{O}(\log n)$, tak nás oprava balančního faktoru stojí $\mathcal{O}(\log n * 1) =$

Časová složitost rotací

Jelikož je hloubka stromu $\mathcal{O}(\log n)$, tak nás oprava balančního faktoru stojí $\mathcal{O}(\log n * 1) = \mathcal{O}(\log n)$

Celková složitost vložení/mazání + oprava balančního faktoru je tedy

Časová složitost rotací

Jelikož je hloubka stromu $\mathcal{O}(\log n)$, tak nás oprava balančního faktoru stojí $\mathcal{O}(\log n * 1) = \mathcal{O}(\log n)$

Celková složitost vložení/mazání + oprava balančního faktoru je tedy

$$\mathcal{O}(\log n + \log n) =$$

Časová složitost rotací

Jelikož je hloubka stromu $\mathcal{O}(\log n)$, tak nás oprava balančního faktoru stojí $\mathcal{O}(\log n * 1) = \mathcal{O}(\log n)$

Celková složitost vložení/mazání + oprava balančního faktoru je tedy

$$\mathcal{O}(\log n + \log n) = \mathcal{O}(\log n)$$

Časová složitost rotací

Jelikož je hloubka stromu $\mathcal{O}(\log n)$, tak nás oprava balančního faktoru stojí $\mathcal{O}(\log n * 1) = \mathcal{O}(\log n)$

Celková složitost vložení/mazání + oprava balančního faktoru je tedy

$$\mathcal{O}(\log n + \log n) = \mathcal{O}(\log n)$$

Složitost vyhledávání zůstává stejná, tedy $\mathcal{O}(\log n)$

Další samovyvažovací datové struktury

Červeno-černé stromy (Red-Black Trees)

- Každý uzel má barvu (červená/černá)
- Používá se v C++ `std::map`, Java `TreeMap`
- Méně rotací než AVL

Další samovyvažovací datové struktury

Červeno-černé stromy (Red-Black Trees)

- Každý uzel má barvu (červená/černá)
- Používá se v C++ `std::map`, Java `TreeMap`
- Méně rotací než AVL

B-stromy

- Uzly mohou mít více než 2 děti
- Používají se v databázích a souborových systémech

Další samovyvažovací datové struktury

Červeno-černé stromy (Red-Black Trees)

- Každý uzel má barvu (červená/černá)
- Používá se v C++ `std::map`, Java `TreeMap`
- Méně rotací než AVL

B-stromy

- Uzly mohou mít více než 2 děti
- Používají se v databázích a souborových systémech

Splay stromy

- Nedávno použité prvky se přesouvají ke kořeni

Shrnutí

Klíčové poznatky

Co jsme se dnes naučili:

Klíčové poznatky

Co jsme se dnes naučili:

- **Zásobník** (LIFO) a **fronta** (FIFO) – základní lineární struktury

Klíčové poznatky

Co jsme se dnes naučili:

- **Zásobník** (LIFO) a **fronta** (FIFO) – základní lineární struktury
- **Spojový seznam** – vkládání $\mathcal{O}(1)$, ale přístup $\mathcal{O}(n)$

Klíčové poznatky

Co jsme se dnes naučili:

- **Zásobník** (LIFO) a **fronta** (FIFO) – základní lineární struktury
- **Spojový seznam** – vkládání $\mathcal{O}(1)$, ale přístup $\mathcal{O}(n)$
- **Binární vyhledávací strom** – vyhledávání v $\mathcal{O}(\log n)$

Klíčové poznatky

Co jsme se dnes naučili:

- **Zásobník** (LIFO) a **fronta** (FIFO) – základní lineární struktury
- **Spojový seznam** – vkládání $\mathcal{O}(1)$, ale přístup $\mathcal{O}(n)$
- **Binární vyhledávací strom** – vyhledávání v $\mathcal{O}(\log n)$
- BST může **degenerovat** – řešením je vyvažování (AVL rotace)

Klíčové poznatky

Co jsme se dnes naučili:

- **Zásobník** (LIFO) a **fronta** (FIFO) – základní lineární struktury
- **Spojový seznam** – vkládání $\mathcal{O}(1)$, ale přístup $\mathcal{O}(n)$
- **Binární vyhledávací strom** – vyhledávání v $\mathcal{O}(\log n)$
- BST může **degenerovat** – řešením je vyvažování (AVL rotace)
- Volba struktury vždy závisí na **operacích**, které potřebujeme

Klíčové poznatky

Strom je ale jen speciální případ obecnější struktury

Klíčové poznatky

Strom je ale jen speciální případ obecnější struktury

Příště nás čekají **grafy** a **hashovací tabulky**